QSC C++17 Wrapper – QSCCPP

Technical Specification · Revision 0.1.0 · November 8, 2026

Quantum Resistant Cryptographic Solutions Corporation

This engineering specification defines the QSCCPP native C++ API, its direct QSC dependency, secure ownership and error contracts, cryptographic algorithms, TLS and certificate abstractions, hardware bridge, configuration, qualification console, examples, and public API declarations.

QSCCPP and QSCSDKCPP are QRCS commercial software. Applicable license: QRCS Commercial Application Software License (QRCS-CASL). The controlling terms of an authorized distribution govern licensing and redistribution.

Contents

1 Foreword 3
2 Introduction and Source Baseline 3
3 References, Definitions and Conventions 5
4 Architecture and C++17 Integration 7
5 Build, Configuration and Deployment 8
6 Type System, Errors and Failure Semantics 9
7 Secure Memory and Utility Functions 10
8 Hash and SHAKE Streaming 11
9 MACs, HKDF and PBMAC1 12
10 Symmetric Ciphers and AEAD 13
11 Entropy, DRBG and Random Services 14
12 Post-Quantum KEMs 15
13 Digital Signatures and Classical Curves 16
14 Explicit Parameter Selection 16
15 X.509 Certificate Inspection and Trust 17
16 PKCS#12, CSR Issuance and CRLs 18
17 TLS 1.3 Socket Architecture 19
18 Hardware Provider Integration 20

19 C++ Examples and Calling Patterns 21
20 Qualification Console and Tests 22
21 Security, Portability and Implementation Risks 24
22 Full Public C++ API Reference 26
23 Build and Release Acceptance Checklist 115
Annex A Types, Classes and Enumerations 116
Annex B Source, Module and Test Inventory 117
Annex C Compatibility Notes and Source Findings 119

1: Foreword

QSCCPP is the QRCS C++17 wrapper for the QSC cryptographic library. It provides typed key arrays, standard-library byte containers, exception-reporting convenience functions, status-based try_ functions, stream-state RAII owners and high-level wrappers for certificate and TLS operation. The companion QSCSDKCPP application is an offline verification console with named examples and fixed-order tests.

Unlike QSCRS, which uses the QSCABI native C interface, QSCCPP includes and calls the QSC C headers directly through extern "C" blocks. Its public C++ types are compiled into the caller's ABI. A C++ application therefore must use a matching QSC source revision, compiler environment and compatible library binary; the wrapper is not a language-neutral stable C ABI layer.

This paper follows the QRCS QSC specification document style with letter-size pages, one-inch margins, Segoe UI body text, Segoe UI Semibold titles and section headings, no bold font, numbered chapters, full paragraph-width tables, and black text except clickable external references. Technical statements are derived from the uploaded QSCCPP(4).zip project, not from undocumented assumptions about a newer native QSC build.

2: Introduction and Source Baseline

2.1 Document scope

QSCCPP supplies approximately three classes of interfaces. Stateless vector functions wrap one-shot cryptographic operations; try_ variants report false or typed native status without throwing. Stateful C++ objects manage QSC contexts and destroy or wipe them through RAII. Higher-level certificate, TLS and hardware objects retain native resources, callback lifetimes and transport configuration with explicitly documented ownership.

A successful C++ build cannot establish runtime algorithm support, cryptographic qualification, peer authentication or hardware presence. The application must validate every parameter length, key policy, authentication result and state transition, and must independently protect stored and imported secrets.

2.2 Snapshot and version baseline

Property	Uploaded-project finding
Source package	QSCCPP(4).zip, containing QSCCPP/ and QSCSDKCPP/ projects.

Property	Uploaded-project finding
Wrapper version	qsccpp_common.h defines qsccpp_version_major=0, minor=1, patch=0; no separate semantic package manifest was included.
Public callable surface	659 publicly callable declarations in 36 contributing headers, counting overloads and non-deleted constructors/destructors/operators.
Core implementation	QSCCPP static C++17 library with 37 listed .cpp translation units and external QSC C headers and library.
Qualification application	QSCSDKCPP executable; 29 named module test calls plus secure_vector lifecycle check (30 sections), direct sequential dispatch.
Build metadata	QSCCPP/CMakeLists.txt and QSCSDKCPP/CMakeLists.txt plus Visual Studio .vcxproj files; no workspace-root CMakeLists.txt or .sln included.
External dependency	QSC native C library and C headers are not bundled in the supplied project archive.
License	QRCS-CASL identified by the companion project source copyright notice; controlling license not reproduced here.
Test evidence	Source files and tests inspected; no QSC compilation, SDK execution or external TLS/hardware exercise performed for this paper.

Table 2.1 Source baseline

2.3 Source of truth and change management

All function signatures in Section 22 are taken from the uploaded public C++ header declarations. They include overloads, class constructors, destructors and public operators, but exclude deleted special member functions. A header declaration may be conditionally compiled, particularly in the hardware namespace, so a function listed here is not always an available runtime operation in every configuration.

The version numbers 0.1.0 in this specification identify the public constants rather than an independently signed release. The QSC native library version, supported algorithm parameters, platform-specific AES acceleration, TLS behavior and certificate semantics are provided by the linked QSC revision and must be qualified together.

3: References, Definitions and Conventions

3.1 Controlling source material

- QSCCPP/qsccpp.h — umbrella include for every public wrapper module.
- QSCCPP/qsccpp_common.h and qsccpp_exception.h — core types, version and typed exceptions.
- QSCCPP/qsccpp_secure.h and qsccpp_utils.h — secure_vector, allocator and byte-vector utilities.
- QSCCPP/qsccpp_sha2.h, qsccpp_sha3.h and qsccpp_shake.h — digest and XOF state.

- QSCCPP/qsccpp_aes.h, qsccpp_chacha.h, qsccpp_csx.h and qsccpp_rcs.h — encryption and AEAD.
- QSCCPP/qsccpp_mlkem.h, qsccpp_mldsa.h, qsccpp_slhdsa.h, qsccpp_ecdh.h and related headers — asymmetric operations.
- QSCCPP/qsccpp_x509.h and qsccpp_x509_issuance.h — certificate, CSR, CRL and PKCS#12 ownership.
- QSCCPP/qsccpp_tls.h and qsccpp_hwprovider.h — TLS and hardware lifecycle contracts.
- QSCSDKCPP/sdkcpp.cpp and sdkcpp.h — ordered qualification entry points and console policy.
- QSCSDKCPP/*_example.cpp — named acceptance checks and illustrative use patterns.
- QSCCPP/CMakeLists.txt and QSCSDKCPP/CMakeLists.txt — C++17 library and executable sources.

3.2 External specifications and reference links

QRCS QSC reference source — https://github.com/Steppenwolfe65/QSC

ISO C++ language reference overview — https://isocpp.org/

CMake build-system reference — https://cmake.org/documentation/

NIST FIPS 197 — AES — https://doi.org/10.6028/NIST.FIPS.197-upd1

NIST FIPS 202 — SHA-3 — https://doi.org/10.6028/NIST.FIPS.202

NIST SP 800-185 — cSHAKE and KMAC — https://doi.org/10.6028/NIST.SP.800-185

NIST FIPS 203 — ML-KEM — https://doi.org/10.6028/NIST.FIPS.203

NIST FIPS 204 — ML-DSA — https://doi.org/10.6028/NIST.FIPS.204

NIST FIPS 205 — SLH-DSA — https://doi.org/10.6028/NIST.FIPS.205

RFC 8446 — TLS 1.3 — https://www.rfc-editor.org/rfc/rfc8446

RFC 5280 — X.509 PKI — https://www.rfc-editor.org/rfc/rfc5280

RFC 5869 — HKDF — https://www.rfc-editor.org/rfc/rfc5869

3.3 Terminology and byte-count conventions

Term	Definition
RAII	Resource Acquisition Is Initialization. Native state is released from a destructor or clear() operation.
byte_vector	qsc::byte_vector = std::vector<std::uint8_t>, caller-owned, ordinary heap storage.
byte_array<N>	qsc::byte_array<N> = std::array<qsc::byte, N>; fixed compile-time width.
secure_vector	Move-only secret byte container with secure byte-erasure on release and reallocation; not OS-locked memory.

Term	Definition
try_ interface	No-throw method returning bool or native typed status; failure does not authorize output consumption.
Opaque native handle	C++ owner holding a QSC heap/state pointer; borrow only during its valid lifetime.
Capacity	Allocated destination length in bytes; distinct from produced or required bytes.
AEAD / AAD	Authenticated encryption with associated data / authenticated but unencrypted payload fields.
FIPS-standard PQC	ML-KEM, ML-DSA and SLH-DSA standards; availability/parameter selection remain build-dependent.
Hardware custody	Policy requiring nonexportable private key material under a qualifying backend; not automatically established by a compile flag.

Table 3.1 Terminology

All array sizes and output lengths in this paper are bytes unless explicitly identified otherwise. A qsc::byte_vector contains binary data; it is not automatically NUL terminated. Key and signature size helpers should be preferred to hardcoded lengths where the selected algorithm or curve supports multiple profiles.

4: Architecture and C++17 Integration

4.1 Wrapper layer and dependency graph

```
C++17 application or SDK qualification executable
        |
        | #include "qsccpp.h"  (standard-library C++ types)
        v
    QSCCPP static C++17 library
        |
        | extern "C" declarations / native QSC C calls
        v
    QSC native C cryptographic library
```

QSCCPP modules are grouped by algorithm families and by state lifetime. Most primitive functions marshal byte vectors and fixed-size key arrays into native QSC buffers, invoke a checked transform or generator, and retain the resulting bytes in caller-owned vectors. Exceptions are confined to the C++ convenience layer; native QSC status values are not silently converted into authentication success.

4.2 Public header organization

The umbrella qsccpp.h includes all public algorithm headers and high-level service headers. Consumers may include a narrower header when only a subset is needed; this does not reduce the linker's requirement for matching QSC implementation objects.

QSCSDKCPP is a separate executable and must not be imported as a cryptographic dependency into applications.

4.3 C++ ABI and binary compatibility

Unlike QSCABI, C++ class layouts, std::vector, std::string, exception types and template instantiations are part of this wrapper's binary-facing contract. C++ object code must be compiled with compatible compiler/CRT, standard-library implementation, architectures, QSC feature macros and native state layout. Distribution across arbitrary C++ ABIs is not assured by a public header.

4.4 Conditional compilation and feature discovery

The hardware module gates provider and key implementations behind QSC_HARDWARE_ENABLED. QSC native compile-time parameter selections also influence which algorithms and implementation paths exist. Build-time enabled does not imply connected hardware, native AES acceleration, an available network peer, or certificate policy acceptance. Call hardware::available for the compiled bridge and check typed statuses for actual operations.

5: Build, Configuration and Deployment

5.1 CMake and Visual Studio project layout

The uploaded QSCCPP/CMakeLists.txt defines add_library(QSCCPP STATIC ...) using C++17 with compiler warnings enabled. It links publicly to ${QSCCPP_QSC_TARGET}. The QSCSDKCPP/CMakeLists.txt defines an executable target linking QSCCPP and expects QSCCPP_QSC_INCLUDE_DIR. A top-level CMake configuration must define the QSC dependency and add both directories in dependency order. The ZIP does not contain that top-level file; therefore these subproject fragments are not a complete independently invokable build.

```
# Illustrative parent-project integration only; no root file shipped:
# add_subdirectory(<external-QSC-source-or-imported-target>)
# set(QSCCPP_QSC_TARGET <matching-QSC-target>)
# set(QSCCPP_QSC_INCLUDE_DIR <matching-QSC-include-path>)
# add_subdirectory(QSCCPP)
# add_subdirectory(QSCSDKCPP)
```

The CMake example is an integration sequence rather than a verified copy-paste command: the actual QSC target and include path must match the user's external

repository and its supported CMake structure. No build files have been added or changed to create this specification.

5.2 Native compiler and CPU profile

QSCCPP requests C++17. Compilers must support the native QSC C include set and the chosen C++ standard library ABI. The QSC cryptographic code may have scalar, AES-NI, AVX/AVX2 or AVX-512 paths. A host without AVX-512 can compile a target selected for it, but cannot safely execute the resulting unsupported ISA instructions; packaging requires target-CPU eligibility checks and aligned QSC/QSCCPP build configurations.

5.3 Windows implementation

The archive includes QSCCPP.vcxproj and QSCSDKCPP.vcxproj plus their filters files. Configure QSC as a separate project or otherwise supply a link-compatible native library; build the native QSC dependency, then QSCCPP, then the SDK console. No .sln file is supplied in the archive, so the source does not establish a complete ready-to-open solution. Verify all additional C dependencies needed by the SSL/TLS, X.509, hardware and asymmetric modules.

5.4 Distribution and reproducibility

Record OS/architecture, compiler and runtime versions, QSC git revision, selected CPU profile, export/static library identity, header hashes, linker settings and the completed sequential test report. The static wrapper itself is not a self-contained redistributable: the external native QSC dependency and its authorized licensing terms must be handled separately. The QRCS-CASL is the commercial license identified in the uploaded SDK source; no alternate product license is inferred.

6: Type System, Errors and Failure Semantics

6.1 Version, status and error enumerations

qsccpp_common.h defines qsc::byte, qsc::byte_vector and qsc::byte_array<N>. It declares qsccpp_version_major=0, qsccpp_version_minor=1, qsccpp_version_patch=0, a status enum, an error_code enum, and the matching string conversion functions. These C++ enumerations are not the same binary types as QSC native qsc_asn1_status, qsc_tls_socket_status or qsc_hw_status.

qsc::status	Meaning / handling
success	Requested wrapper operation completed according to its contract.

qsc::status	Meaning / handling
failure	Unclassified operation failure; do not consume output.
invalid_argument	Pointer, parameter or mode does not meet the documented contract.
invalid_size	Input/output length or parameter-set width is not acceptable.
authentication_failure	Ciphertext authentication or other verifier rejected input.
allocation_failure	C++ destination/state allocation failed.
qsc_failure	Underlying native QSC operation reported failure.

Table 6.1 Public result categories

6.2 Exceptions versus no-throw variants

The crypto_exception class stores a location, origin, human-readable message and typed error_code. Its what(), code(), location(), origin() and message() methods expose that diagnostic state. A throwing convenience function may be useful in application initialization; use try_ methods and explicit native status handling on latency-sensitive protocol paths where exceptions would complicate cleanup. Do not interpret a false result as authenticated plaintext or a valid signature.

6.3 Result-buffer ownership

Unless an API explicitly documents special ownership, a byte_vector output is caller-owned and populated by the wrapper. A failed try_ call may clear or leave output unchanged depending on the header contract; consume output only after success. The output capacity for opaque native methods such as hardware sign or PKCS#12 export may be supplied by the caller, and returned lengths must be checked independently of buffer capacity.

6.4 Allocator equality and security comparisons

secure_byte_allocator::operator== deliberately returns true because allocator instances are stateless and interchangeable; its equality is an allocator compatibility contract, not a comparison of secret bytes. Never use this operator to test key equality, ciphertext authenticity or MAC equality. Use an appropriate cryptographic verification method or a secure content-comparison utility with equal-length buffers.

7: Secure Memory and Utility Functions

7.1 secure_vector ownership

secure_vector is final, move-only, and backed by std::vector<byte, secure_byte_allocator>. Its allocator wipes allocated byte storage before release,

including on reallocation. The class exposes data(), size(), empty(), resize() and clear(), and supports move construction/assignment but disables copying. A moved-from secure_vector remains valid but empty; applications must not retain raw pointers across resize or move.

The secure allocator does not lock pages into RAM, prevent paging, sanitize copies outside the vector, or guarantee that unrelated compiler-generated temporaries are erased. Concurrent mutation of one vector is not supported without caller synchronization. Ordinary byte_vector is not a secret-aware container and should not be used for long-term private key storage unless the application explicitly manages erasure.

7.2 Memory utilities and failure containment

qsc::utils::data_or_null converts empty vectors/strings to a pointer suitable for zero-length native calls, and qsc::utils::resize reports allocation success. qsc::utils::secure_clear clears a byte_vector using the QSC secure erasure operation. The is_equal utility must be read as its implemented byte-content comparison, not as authenticated message verification. The application must separately enforce expected lengths and secrecy requirements.

```
qsc::secure_vector key_material(32U);
// Populate from a validated trusted provider.
qsc::byte_vector public_input = { 0x51U, 0x53U, 0x43U };
qsc::byte_vector temporary;
// Handle return status from every cryptographic operation.
key_material.clear();
```

7.3 RAII and error paths

An object destructor must be safe on a cleared or failed object whenever documented, and should run on every return or exception path. The higher-level TLS and X.509 wrappers expose native() borrowing accessors; callers must not free their returned pointers, save them beyond owner lifetime, or mutate them while concurrent operations are running.

8: Hash and SHAKE Streaming

8.1 One-shot SHA2 and SHA3 interfaces

sha2::compute256/compute384/compute512 and the SHA3-family compute interfaces return byte vectors from native digests. Their try_compute counterparts accept caller-

owned output containers or pointer/length forms. Digest lengths are determined by the selected algorithm; do not assume message encoding or a NUL terminator. Message digests alone provide no keyed authenticity.

```
const qsc::byte_vector message = { 0x51U, 0x53U, 0x43U };
qsc::byte_vector digest;
if (qsc::sha2::try_compute256(message, digest))
{
    // digest contains SHA2-256 output; caller still owns it.
}
```

8.2 Incremental digest objects

qsc::sha2::stream256/stream384/stream512 and matching SHA3 streams hold non-copyable QSC digest state, expose reset(), try_update(), try_finalize(), finalize() and active(), and wipe native state at destruction. A finalized stream must be reset before it can absorb further data. Stateful instances are not documented as safe for concurrent use.

8.3 SHAKE XOF state and absorb/finalize/squeeze

The SHAKE family supports variable-length outputs, including stateful XOF operations. A state must be initialized for the selected rate/strength, absorb input before finalization, then squeeze the requested number of bytes. Output length is a protocol-level parameter and is not implied by a single digest-size constant. Callers must ensure domain separation and output-length agreement.

9: MACs, HKDF and PBMAC1

9.1 HMAC, KMAC, QMAC and authentication

The wrapper groups HMAC-SHA2, KMAC variants, QMAC and PBMAC1 under C++ vector-based interfaces. Where supported, functions have throwing and try_ alternatives. A MAC authenticates the exact protocol-defined byte sequence under the correct secret key. Digest equality without a keyed verifier is insufficient; the application must reject mismatched authentication values and wrong key/parameter profiles.

9.2 HKDF and derived-key separation

HKDF extract/expand wrappers require explicit salt/input key material, info and requested output. Salt and context/info are not interchangeable. An application should identify purpose, peer identity, algorithm profile and protocol transcript through a

stable derivation contract; each role and algorithm should receive distinct derived key material. Clean any temporary secrets after key installation.

9.3 PBMAC1 password-based operations

PBMAC1 compute/verify/derive interfaces have explicit digest selection, password salt, work factor and key-output sizes in the published header. Do not use a fabricated universal iteration count: deployment-specific resource constraints and threat models control password hardness. Authentication failures must not reveal key-dependent partial results.

10: Symmetric Ciphers and AEAD

10.1 Cipher families

Wrapper family	Public interface	Security requirement
AES CTR	ctrbe/ctrle transform functions	Encryption and decryption are symmetric; counters cannot repeat with a key.
AES CBC PKCS#7	try_cbc256_encrypt/decrypt	No native authentication; require independent integrity protection.
AES GCM	try_gcm128/256_encrypt/decrypt	Unique nonce, bounded AAD, verified appended tag before releasing plaintext.
AES HBA	try_hba256_encrypt/decrypt	QSC authenticated envelope; enforce exact mode, key size and tag.
ChaCha20	chacha::transform/try_transform	Stream confidentiality only; authentication required separately.
ChaCha20-Poly1305	chacha::poly1305_*	AEAD; fresh nonce, MAC verification and ciphertext/tag format.
CSX and RCS	encrypt/decrypt or try_ variants	Proprietary QSC authenticated cipher formats and key/nonce policy.

Table 10.1 Symmetric operation families

10.2 Authenticated ciphertext and buffer lengths

AES GCM encryption appends a 16-byte authentication tag in the described C++ interface. AES HBA and RCS/CSX have their QSC-defined tag lengths; those lengths must be accounted for in the allocated output and in the receiving protocol. Decrypt functions must verify the tag before the plaintext is consumed. A nonthrowing method may clear output on authentication failure; the caller should treat it as absent regardless of prior contents.

10.3 Overlap and replay policy

Do not assume every wrapper operation accepts overlapping source and destination buffers. The vector-returning public API simplifies out-of-place use, while specialized pointer overloads and native functions require their header-specific overlap contracts. CTR and stream modes require nonce/counter uniqueness. AEAD also requires the application to implement record ordering, replay prevention and identity binding where the protocol demands them.

10.4 Representative call sequence

```
qsc::aes::aes256_key key{};
qsc::aes::gcm_nonce nonce{};  // MUST be populated uniquely
qsc::byte_vector aad;
qsc::byte_vector plaintext;
qsc::byte_vector ciphertext;
const bool ok = qsc::aes::try_gcm256_encrypt(
    key, nonce, plaintext, aad, ciphertext);
// A real application supplies secret key material, unique nonce,
// and validated inputs before calling the wrapper.
```

11: Entropy, DRBG and Random Services

11.1 Entropy adapters and availability

The entropy namespace contains source selectors and is_available, provider_to_string, try_generate and generate APIs. The QSC backend determines whether a random source is usable on the target OS. Source availability and cryptographic entropy certification are separate matters. Invalid or failed provider operations must not fall back to deterministic zeroes or simulation data in production.

11.2 secrand and DRBG state

The secrand convenience layer manages initialization, generation and disposal. The drbg::hcg and drbg::csg classes own non-copyable, non-movable native states and expose initialize(), try_generate(), try_reseed(), initialized() and clear(). They require trusted seed bytes and do not themselves create independent entropy evidence. The native implementation may request additional QSC entropy during generation according to its options.

11.3 Seed and reseed constraints

The header for HCG specifies a 64-byte trusted initialization seed; CSG accepts 32- or 64-byte seeds. Re-seeding must use independent, trusted new entropy and cannot be replaced by predictable test vectors. These contexts are not thread-safe; a single application owner or synchronization wrapper must serialize access and disposal.

12: Post-Quantum KEMs

12.1 Supported families and parameter choices

The public QSCCPP KEM namespaces include mlkem, hqc and mceliece, exposing algorithm_name and key/ciphertext/secret size selection, keypair, encapsulation and decapsulation APIs. Some modules define parameter_set enumerations rather than relying on a single compile-time native selection. A requested profile must be validated against the linked QSC build and must be negotiated within an authenticated protocol.

12.2 Keypair and encapsulation lifecycle

```
parameter_set/profile -> size helpers -> keypair generation
        -> encapsulate(peer public key)
        -> transmit encapsulation + protocol transcript
peer private key -> decapsulate(ciphertext)
        -> independent key confirmation/authentication
```

A valid KEM shared secret does not identify the peer or validate a public-key origin. Some KEMs intentionally return derived bytes on malformed ciphertext through implicit rejection. Applications must authenticate the handshake transcript, bind the selected algorithm/parameter to the peer identity and avoid revealing decryption-validity side channels.

12.3 Memory and error handling

Private keys and shared secrets should use secure_vector wherever the public API offers it. Keypair structures and parameter-set helpers vary by algorithm; do not reinterpret one key format as another. Reject mismatched sizes before a native call and wipe temporary secret data on failure or after key derivation.

13: Digital Signatures and Classical Curves

13.1 Post-quantum signatures

The wrapper exposes mldsa, falcon, sphincsplus and slhdsa signature namespaces. Implementors must distinguish attached signing from detached signatures and preserve the requested algorithm profile in both signature and key formats. A successful verification proves that a message verifies under the supplied public key; it does not grant certificate trust or application authorization.

13.2 Classical key agreement and signatures

The ecdh, eddh, ecdsa and eddsa namespaces provide curve selectors, size helpers, key generation, public-from-private and sign/verify or shared secret operations. Their public function naming is not equivalent to the full native QSC low-level API. Use the declared C++ wrapper signature for byte-vector and secure-vector arguments, and document the message representation used by each signing scheme.

13.3 Scalar keys, seeds and representations

The C++ curve wrappers add explicit scalar signing and public-key derivation methods in their newer interfaces. Imported private scalar bytes require strict range validation and key custody rules. The wire representation for ECDSA may be raw or encoded depending on the chosen wrapper/native API; do not assume that attached signed-message bytes can be substituted into an external DER-based protocol without conversion and test evidence.

14: Explicit Parameter Selection

14.1 Algorithm profile selection and validation

Several PQC and curve namespaces supply parameter_set or curve enum classes. The wrapper avoids exposing native QSC compile-time macro names as dynamic selectors in every case. When a selector is present, obtain size metadata from the same selected profile and check actual call success. A selection absent from the linked QSC object files must fail, not silently substitute an adjacent profile.

14.2 SLH-DSA context and prehash

slhdsa.h documents parameter_set and prehash selectors. The FIPS 205 family has context and prehash policy requirements; use the exact context byte sequence and do

not assume a generic SHA-2 digest can replace a specified prehash scheme. Reject unsupported profile combinations and never confuse a public-key size query with signature capacity.

15: X.509 Certificate Inspection and Trust

15.1 Owned certificate and borrowed native pointers

qsc::x509::certificate is move-aware and owns heap-stable native certificate state with a retained DER copy. QSC parsing can borrow pointers into DER input, so the wrapper protects against dangling storage by preserving its own bytes until clear/destruction. native() returns a borrowed pointer, not transferable ownership. A moved-from certificate is unloaded and must not be used as a validated peer identity.

15.2 Identity accessors and hostname checks

Certificate subject, issuer, common-name and DNS-name extraction methods expose decoded identity fields. These checks are useful but not sufficient for authentication: first establish a valid certificate path to an independently trusted issuer and apply time, key usage, purpose, hostname and revocation policy. A successful DER parse or matching common name is not evidence of trusted issuance.

15.3 Trust store policy

The trust_store object controls independently trusted anchors and native verification context. Trust anchors must be explicitly provisioned, not inferred from the untrusted peer chain. Do not convert every parse or signature verification result to an overall authorization decision without checking the specific native verification status.

16: PKCS#12, CSR Issuance and CRLs

16.1 CSR generation and certificate issuance

The qsccpp_x509_issuance.h module supplies certificate_request, a certificate issuance helper and CSR verification through QSC. CSR signature validity shows possession or control of the submitting key, not permission to issue the requested subject or DNS identities. Every issuance request requires external policy authorization, correctly scoped issuer trust, serial-number management and validity rules.

16.2 CRL builder and signature verification

crl_builder is a move-only RAII owner that configures issuer, update times and signature algorithm, adds serial numbers and signs DER through a caller-supplied native callback. It verifies signed CRLs only relative to the supplied issuer; issuer trust-path validation remains caller-owned. A duplicate serial, invalid time interval or missing CA/cRLSign authority is a failure rather than a revocation clearance.

16.3 PKCS#12 import and secret handling

The pkcs12_bundle class owns parsed native PKCS#12 state, including private-key material. The public header indicates a password/MAC-protected decode requirement and provides controlled accessors. Clear native state and every application-owned password/private-key copy after use. A successful container import does not automatically grant trust in an embedded certificate chain.

16.4 C callback boundaries

Certificate issuance, CRL signing and signature verification use plain C function-pointer callbacks, passed together with caller-managed context. These callbacks must not throw across native C frames, must return native status on failure, and must not retain pointers to transient C++ objects beyond the synchronous call. Key-handling and trust validation remain separate from correct DER serialization.

17: TLS 1.3 Socket Architecture

17.1 Object and ownership model

Class or type	Purpose	Lifecycle contract
qsc::tls::context	Role-specific trust, socket, cipher suite and authentication policy	RAII; cannot be copied or moved.
qsc::tls::connection	Connected protocol state, handshake result, ALPN and protected I/O	RAII; state is invalid before successful connect/accept.
qsc::tls::listener	Accepts server-side socket sessions	Own listener resources and closes on clear/destruction.
qsc::tls::server	Blocking/concurrent managed server loop	Owns server state; bound callback lifetimes must be respected.
qsc::tls::peer_info	Copied certificate identity and negotiated security metadata	Snapshots; not independent trust verification.
qsc_tls_socket_options / ticket_policy	Native configuration structs returned by helper functions	Caller configures before a handshake/session starts.

Table 17.1 QSC TLS C++ facade

17.2 Configuration and handshake sequence

```
qsc::tls::context ctx;
ctx.set_default_client_policy();
ctx.set_strict_policy();
// Load approved trust anchors and set SNI/ALPN before connecting.
// Create and connect an owned qsc::tls::connection.
// Require authentication / hostname checks before application I/O.
// Stop the session and release RAII objects on completion.
```

Methods expose both checked try_ setters and throwing configuration operations. A context configured for a client is not a completed TLS connection; the handshake and trust/hostname checks must succeed before protected application data is exchanged. Configure any client certificates, server identities, named groups, ALPN, session tickets and transport timeouts before network use.

17.3 TLS snapshot does not imply live success

The supplied tls_example.cpp tests an initialized but disconnected connection object and expects is_connected()==false, is_handshake_complete()==false and no selected ALPN. The success printed by that section means the unconnected-state contract passed; it does not indicate a successful TLS session or network handshake. Live multi-host testing remains a separate qualification task.

17.4 Peer authentication and I/O

The peer_info structure copies negotiated cipher suite, group, signature scheme, identity strings and flags for authentication, hostname matching, chain validity, PSK and early-data outcomes. Treat these fields as snapshots of native state, not authorization policies. Higher-layer framing, read/write partial results, cancellation and shutdown sequencing remain application responsibilities.

17.5 Callbacks and concurrency

Server logging, client authorization and ticket PSK lookups are callback-based. The callback state must remain valid until all native invocations have finished. Client-authorization callbacks must be fail-closed, and the application must synchronize all shared callback state. For concurrent listeners, resource shutdown must prevent further callbacks before destroying their context.

18: Hardware Provider Integration

18.1 Optional hardware bridge

hardware::available() reports whether the QSC_HARDWARE_ENABLED bridge is compiled. The provider/key interfaces below this feature guard are absent when the guard is disabled. Compilation alone does not establish that a TPM, HSM, PKCS#11 module, secure element or qualified attestation device is attached; device-specific open and operation statuses are mandatory.

18.2 Provider and key ownership

hardware::provider and hardware::key are move-only RAII wrappers backed by a shared provider_state. A key keeps the native provider alive until its transient handle is released. The backend_context is caller-owned and must remain valid until the last key releases it. Closing a provider with live keys reports busy. Public-key extraction and device-attestation evidence are separate from private key custody; hardware private-key export fallback is not implemented.

18.3 Fail-closed key policies

default_key_policy constructs a version/size-qualified key policy requiring hardware custody by default. Operations such as key generation, open_key, sign, agree, random and attest return typed qsc_hw_status outcomes. Attestation evidence must be independently verified, and an exported derived secret must meet the backend's export policy. Backend callback exceptions and concurrent disposal must be prevented by the application.

19: C++ Examples and Calling Patterns

19.1 Source examples and test harness

QSCSDKCPP includes independently named source files for SHA2/SHA3/SHAKE, encoding, keyed MACs, symmetric encryption, entropy, public-key functions, X.509, TLS, hardware and expanded API tests. The C++ executable calls each module through direct named statements and prints a standardized QRCS information header, CPU/intrinsics guidance, individual PASS/FAIL results and a final summary.

These files are executable qualification examples rather than a library header API. They may depend on QSC-specific known-answer fixtures and native features. No external

peer, PKI trust anchor, physical hardware token or native processor ISA can be presumed available just because the test source exists.

19.2 Representative integration paths

Task	Source file	Steps to reproduce in an application
AES-GCM	aes_example.cpp	Construct key and unique nonce, set AAD, encrypt, verify decryption and tampering.
SHA-2 streaming	sha2_example.cpp	Initialize state, update fragments, finalize and compare expected digest.
ML-KEM	mlkem_example.cpp	Select parameter set, generate keys, encapsulate/decapsulate and compare secrets.
ECDSA	ecdsa_example.cpp	Generate/derive curve key, sign, verify and reject modified signature.
X.509	x509_example.cpp	Parse certificate, inspect identity, validate selected trust policy.
TLS	tls_example.cpp	Exercise context, socket options, unconnected connection and policy lifecycle.
Hardware	hardware_example.cpp	Check compiled bridge, provider lifecycle and optional backend behavior.
API extensions	extension_example.cpp	Exercise explicit curves/signatures, PBMAC1, DRBG and X.509 issuance/CRL.

Table 19.1 Implementor example map

19.3 Interoperability and negative tests

For every chosen cryptographic primitive, add separate tests for invalid length, invalid authentication tag, mismatched key, unsupported parameter set, state after finalize, and secret-memory cleanup. Testing only positive roundtrips can conceal accidental self-consistency between a mistaken producer and consumer. Exchange known-answer or externally specified data when interoperability matters.

20: Qualification Console and Tests

20.1 Sequential test architecture

The sdkcpp.cpp runner invokes each test module using direct statements rather than switch dispatch, a registry or test-invocation loop. It includes a secure buffer shrink/grow/move/clear contract and 29 named module calls, resulting in 30 distinct sections. The executable prints CPU capability information and a recommended native intrinsics profile; those labels are advisory and do not select a new native QSC binary at runtime.

20.2 Interactive and CI console behavior

The entry points run_all(), run_console() and run_command_line(argc, argv) separate test execution from the console banner, summary and optional close prompt. The documented --no-pause, -nopause and /nopause options avoid a blocking prompt in noninteractive CI. Invalid arguments have an explicit error exit. The static example code must be rebuilt with the intended QSC and C++ wrapper sources before its results can be used as release evidence.

20.3 Actual test scope and limits

The existing TLS test focuses on non-network state, configuration and object lifetime and explicitly avoids opening a live socket. The hardware suite may skip actual physical token work when no qualifying backend is compiled/attached. Correctness of cryptographic algorithms, certificate validation, full TLS interoperability and native hardware custody must be tested separately on supported platforms. No tests were run during this document-generation task.

20.4 Test group inventory

Direct test entry point	Scope / category
secure_buffer_contract	Secure memory lifecycle, grow/shrink, move, clear
encoding_run_all	Encoding named test group
sha2_run_all	Sha2 named test group
sha3_run_all	Sha3 named test group
shake_run_all	Shake named test group
hmac_run_all	Hmac named test group
kmac_run_all	Kmac named test group
hkdf_run_all	Hkdf named test group
qmac_run_all	Qmac named test group
aes_run_all	Aes named test group
chacha_run_all	Chacha named test group
csx_run_all	Csx named test group
rcs_run_all	Rcs named test group
scb_run_all	Scb named test group
entropy_run_all	Entropy named test group
secrand_run_all	Secrand named test group
mlkem_run_all	Mlkem named test group
mldsa_run_all	Mldsa named test group
falcon_run_all	Falcon named test group
sphincsplus_run_all	Sphincsplus named test group
hqc_run_all	Hqc named test group
mceliece_run_all	Mceliece named test group

Direct test entry point	Scope / category
ecdh_run_all	Ecdh named test group
eddh_run_all	Eddh named test group
ecdsa_run_all	Ecdsa named test group
eddsa_run_all	Eddsa named test group
x509_run_all	X509 named test group
extension_run_all	Extension named test group
hardware_run_all	Hardware named test group
tls_run_all	Tls named test group

Table 20.1 Directly invoked qualification sections

21: Security, Portability and Implementation Risks

21.1 Key and nonce management

Maintain separation between long-term private keys, KEM shared secrets, derived traffic keys and nonsecret public data. Use secure_vector for private keys or secrets when the API supports it; clear every sensitive copy after use. Ensure nonce/counter uniqueness under a cipher key, reject repeated or downgraded parameter selections, and implement transcript/peer identity authentication outside primitive KEM roundtrips.

21.2 Certificate issuance and trust

The C++ X.509 and CSR interfaces can parse and sign material but cannot independently establish organizational authorization, required name constraints or trusted roots. Validate issuer signing authority and certificate path before treating signed CRLs or generated CSRs as grounds for acceptance. Enforce external authorization for issuance to avoid accidental local CA privilege escalation.

21.3 State lifetime and thread discipline

RAII prevents many leaks but does not make an object thread-safe. Shared TLS, hardware, secret vector, hash and DRBG objects require strict lifetime sequencing. Avoid borrowing native() state into another thread after the owner can move, clear or destruct. Native callbacks must not throw across C entry points. Release hardware keys before provider destruction; maintain callback state beyond native operation completion.

21.4 Cross-platform build profile and evidence

The native QSC library may require different scalar/intrinsic builds by CPU. Always run the selected binary only on supported hardware and verify that the C++ wrapper links an appropriate QSC revision. A CMake source list or Visual Studio project file is not

evidence of a passing build. Release notes should record the actual compiler commands, platform, processor features, build diagnostics, fixture provenance and named PASS/FAIL results.

21.5 Known source-package limitations

- No top-level CMake project file or Visual Studio solution appears in the ZIP; the two child project definitions depend on external configuration.
- QSC native headers and library are external and not provided in the uploaded archive.
- The C++ wrapper uses the native QSC API rather than the separately versioned QSCABI and has no independent C ABI compatibility guarantee.
- The hardware provider class and related functions are conditionally compiled.
- The SDK TLS connection-state example expects unconnected state; it does not execute a live client/server handshake.
- Hardware attestation evidence is not independently verified merely by obtaining signed device evidence.
- The stateless secure allocator equality operators cannot be used for cryptographic content comparisons.

22: Full Public C++ API Reference

This section reproduces 659 callable C++ declarations inventoried in 36 public project headers. The count includes overloads, public constructors/destructors and selected inline public operators, and excludes deleted special members and private methods. Entries are organized by source header and show source-derived Doxygen descriptions where provided. Types and exception/status values retain their names; conditional compilation and external native QSC availability remain applicable.

The original header signatures are intentionally preserved. Some long template/parameter declarations wrap to the next line in Word without altering the API semantics. Exact function implementation is in the corresponding QSCCPP/*.cpp file; header contracts are authoritative for parameter types, return types, constness and noexcept declarations.

22.1 qsccpp_aes.h — 15 callable declarations

Public aes C++ functions and object methods.

qsc::aes::ctrbe256_transform

byte_vector ctrbe256_transform(const aes256_key& key, const aes_iv& nonce, const byte_vector& input);

Purpose: Public ctrbe256 transform operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::aes::gcm256_encrypt

byte_vector gcm256_encrypt(const aes256_key& key, const gcm_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data);

Purpose: Public gcm256 encrypt operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::aes::gcm256_decrypt

byte_vector gcm256_decrypt(const aes256_key& key, const gcm_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data);

Purpose: Public gcm256 decrypt operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::aes::try_ctrbe128_transform

bool try_ctrbe128_transform(const aes128_key& key, const aes_iv& nonce, const byte_vector& input, byte_vector& output) noexcept;

Purpose: Transform AES-128 CTR big-endian data; encrypt/decrypt use identical operation

Parameters: key: AES-128 key; nonce: Caller-supplied unique 16-byte counter; input: Input bytes.; output: Output bytes; cleared on allocation failure.

Return: True on success

qsc::aes::try_ctrle128_transform

bool try_ctrle128_transform(const aes128_key& key, const aes_iv& nonce, const byte_vector& input, byte_vector& output) noexcept;

Purpose: Transform AES-128 little-endian counter mode

Parameters: key: AES-128 key; nonce: Unique 16-byte counter; input: Input; output: Output.

Return: True on success. CTR counters must not be reused under the same key

qsc::aes::try_ctrle256_transform

bool try_ctrle256_transform(const aes256_key& key, const aes_iv& nonce, const byte_vector& input, byte_vector& output) noexcept;

Purpose: Transform AES-256 little-endian counter mode

Parameters: key: AES-256 key; nonce: Unique counter; input: Input; output: Output.

Return: True on success. CTR is unauthenticated

qsc::aes::try_gcm128_encrypt

bool try_gcm128_encrypt(const aes128_key& key, const gcm_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, byte_vector& ciphertext) noexcept;

Purpose: Authenticated AES-128 GCM encryption with a 16-byte appended tag

Parameters: key: Secret 16-byte key; nonce: Unique 12-byte nonce; plaintext: Input.; associated_data: AAD, at most QSC_GCM_MAXAAD_SIZE bytes; ciphertext: Output.

Return: True on success; false on overflow, oversize AAD, or allocation failure

qsc::aes::try_gcm128_decrypt

bool try_gcm128_decrypt(const aes128_key& key, const gcm_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, byte_vector& plaintext) noexcept;

Purpose: AES-128 GCM decrypt and verify appended authentication tag before release

Parameters: key: Secret key; nonce: Original nonce; ciphertext: Ciphertext and appended tag.; associated_data: Original AAD; plaintext: Output (cleared on authentication failure).

Return: True for authenticated ciphertext; false otherwise

qsc::aes::try_hba256_encrypt

bool try_hba256_encrypt(const aes256_key& key, const aes_iv& nonce, const byte_vector& plaintext, const byte_vector& associated_data, byte_vector& ciphertext) noexcept;

Purpose: AES-256 HBA encrypt with appended QSC_HBA256_MAC_SIZE authentication tag

Parameters: key: AES-256 key; nonce: Fresh 16-byte nonce; plaintext: Nonempty data.; associated_data: AAD of at most QSC_HBA_MAXAAD_SIZE; ciphertext: Authenticated result.

Return: True on success. Rejects oversized inputs and clears output on failure

qsc::aes::try_hba256_decrypt

bool try_hba256_decrypt(const aes256_key& key, const aes_iv& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, byte_vector& plaintext) noexcept;

Purpose: Authenticate then decrypt AES-256 HBA ciphertext

Parameters: key: AES-256 key; nonce: Original nonce; ciphertext: Includes 32-byte tag.; associated_data: Original AAD; plaintext: Authenticated result, cleared on failure.

Return: True on verified ciphertext; false otherwise

qsc::aes::try_cbc256_encrypt

bool try_cbc256_encrypt(const aes256_key& key, const aes_iv& iv, const byte_vector& plaintext, byte_vector& ciphertext) noexcept;

Purpose: AES-256 CBC PKCS7 encrypt (unauthenticated; use only with separate MAC)

Parameters: key: Key; iv: Fresh 16-byte IV; plaintext: Nonempty input; ciphertext: Output.

Return: True on success; false on allocation/overflow failure

qsc::aes::try_cbc256_decrypt

bool try_cbc256_decrypt(const aes256_key& key, const aes_iv& iv, const byte_vector& ciphertext, byte_vector& plaintext) noexcept;

Purpose: AES-256 CBC PKCS7 decrypt (no ciphertext authentication)

Parameters: key: Key; iv: Original IV; ciphertext: Padded data; plaintext: Output.

Return: False for invalid padding or empty recovered content. Use AEAD for untrusted inputs

qsc::aes::try_ctrbe256_transform

bool try_ctrbe256_transform(const aes256_key& key, const aes_iv& nonce, const byte_vector& input, byte_vector& output) noexcept;

Purpose: Non-throwing ctrbe256 transform interface. Check the result before reading any output.

qsc::aes::try_gcm256_encrypt

bool try_gcm256_encrypt(const aes256_key& key, const gcm_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, byte_vector& ciphertext) noexcept;

Purpose: Non-throwing gcm256 encrypt interface. Check the result before reading any output.

qsc::aes::try_gcm256_decrypt

bool try_gcm256_decrypt(const aes256_key& key, const gcm_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, byte_vector& plaintext) noexcept;

Purpose: Non-throwing gcm256 decrypt interface. Check the result before reading any output.

22.2 qsccpp_chacha.h — 6 callable declarations

Public chacha C++ functions and object methods.

qsc::chacha::transform

byte_vector transform(const chacha256_key& key, const chacha_nonce& nonce, const byte_vector& input);

Purpose: Invoke the native QSC-backed transform operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::chacha::poly1305_encrypt

byte_vector poly1305_encrypt(const chacha256_key& key, const chacha_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data);

Purpose: Public poly1305 encrypt operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::chacha::poly1305_decrypt

byte_vector poly1305_decrypt(const chacha256_key& key, const chacha_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data);

Purpose: Public poly1305 decrypt operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::chacha::try_transform

bool try_transform(const chacha256_key& key, const chacha_nonce& nonce, const byte_vector& input, byte_vector& output) noexcept;

Purpose: Non-throwing transform interface. Check the result before reading any output.

qsc::chacha::try_poly1305_encrypt

bool try_poly1305_encrypt(const chacha256_key& key, const chacha_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, byte_vector& ciphertext) noexcept;

Purpose: Non-throwing poly1305 encrypt interface. Check the result before reading any output.

qsc::chacha::try_poly1305_decrypt

bool try_poly1305_decrypt(const chacha256_key& key, const chacha_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, byte_vector& plaintext) noexcept;

Purpose: Non-throwing poly1305 decrypt interface. Check the result before reading any output.

22.3 qsccpp_common.h — 2 callable declarations

Core version, status and error-code helpers.

qsc::status_to_string

const char* status_to_string(status value) noexcept;

Purpose: Public status to string operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::error_code_to_string

const char* error_code_to_string(error_code value) noexcept;

Purpose: Public error code to string operation; review the declared signature and its QSC-backed implementation for preconditions.

22.4 qsccpp_csx.h — 4 callable declarations

Public csx C++ functions and object methods.

qsc::csx::encrypt

byte_vector encrypt(const csx_key& key, const csx_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, const byte_vector& info);

Purpose: Invoke the native QSC-backed encrypt operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::csx::decrypt

byte_vector decrypt(const csx_key& key, const csx_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, const byte_vector& info);

Purpose: Invoke the native QSC-backed decrypt operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::csx::try_encrypt

bool try_encrypt(const csx_key& key, const csx_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, const byte_vector& info, byte_vector& ciphertext) noexcept;

Purpose: Non-throwing encrypt interface. Check the result before reading any output.

qsc::csx::try_decrypt

bool try_decrypt(const csx_key& key, const csx_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, const byte_vector& info, byte_vector& plaintext) noexcept;

Purpose: Non-throwing decrypt interface. Check the result before reading any output.

22.5 qsccpp_drbg.h — 14 callable declarations

Non-copyable HCG/CSG DRBG lifetime and generation methods.

qsc::drbg::hcg::hcg

hcg() noexcept;

Purpose: Initialize inert HCG storage

qsc::drbg::hcg::~hcg

~hcg();

Purpose: Erase any HCG internal key, nonce and counter state

qsc::drbg::hcg::initialize

bool initialize(const secure_vector& seed, const byte_vector& info, bool prediction_resistance) noexcept;

Purpose: Initialize with exactly 64 bytes from a trusted entropy source

Parameters: seed: Secret 64-byte key material.; info: Optional personalization up to 56 bytes.; prediction_resistance: Request native ACP reseeding on generation.

Return: True if native initialization yielded an active HCG state

qsc::drbg::hcg::try_generate

bool try_generate(std::size_t size, secure_vector& output) noexcept;

Purpose: Generate nonzero length output in secure memory

Parameters: size: Requested bytes; output: Secure destination, erased on failure.

Return: True only if the native generator succeeds; disposes on failure

qsc::drbg::hcg::try_reseed

bool try_reseed(const secure_vector& seed) noexcept;

Purpose: Incorporate additional seed material; does not substitute for entropy checks

Parameters: seed: Nonempty trusted new secret seed.

Return: True if state was active

qsc::drbg::hcg::initialized

bool initialized() const noexcept;

Purpose: Return true if the native state is initialized

qsc::drbg::hcg::clear

void clear() noexcept;

Purpose: Erase all generator state, returning to an inert object

qsc::drbg::csg::csg

csg() noexcept;

Purpose: Initialize inert CSG storage

qsc::drbg::csg::~csg

~csg();

Purpose: Erase all retained Keccak state and cache material

qsc::drbg::csg::initialize

bool initialize(const secure_vector& seed, const byte_vector& info, bool prediction_resistance) noexcept;

Purpose: Initialize with exactly 32 or 64 secret bytes

Parameters: seed: Trusted seed.; info: Optional cSHAKE personalization.; prediction_resistance: Request periodic native ACP injection.

Return: True only if QSC established a valid Keccak rate

qsc::drbg::csg::try_generate

bool try_generate(std::size_t size, secure_vector& output) noexcept;

Purpose: Generate nonzero-length securely held random output

Parameters: size: Desired byte count.; output: Erased on failure.

Return: True when native generator returns success

qsc::drbg::csg::try_reseed

bool try_reseed(const secure_vector& seed) noexcept;

Purpose: Mix additional nonempty secret seed into active CSG state

Parameters: seed: Trusted new seed.

Return: False if uninitialized or empty

qsc::drbg::csg::initialized

bool initialized() const noexcept;

Purpose: Return true if active state is present

qsc::drbg::csg::clear

void clear() noexcept;

Purpose: Securely erase and deactivate generator state

22.6 qsccpp_ecdh.h — 23 callable declarations

Public ecdh C++ functions and object methods.

qsc::ecdh::public_key_size

std::size_t public_key_size(curve selected) noexcept;

Purpose: Return curve-specific key lengths in bytes; zero for unsupported values

Parameters: selected: Explicit ECDH curve.

Return: Public key byte count

qsc::ecdh::private_key_size

std::size_t private_key_size(curve selected) noexcept;

Purpose: Return curve-specific private scalar length

Parameters: selected: Explicit ECDH curve.

Return: Private scalar byte count

qsc::ecdh::shared_secret_size

std::size_t shared_secret_size(curve selected) noexcept;

Purpose: Return curve-specific shared secret length

Parameters: selected: Explicit ECDH curve.

Return: Agreement output byte count

qsc::ecdh::seed_size

std::size_t seed_size(curve selected) noexcept;

Purpose: Return curve-specific deterministic seed size

Parameters: selected: Explicit ECDH curve.

Return: Required seed bytes

qsc::ecdh::try_generate_keypair

bool try_generate_keypair(curve selected, keypair& output) noexcept;

Purpose: Generate a keypair for the requested curve with QSC CSP entropy

Parameters: selected: Explicit ECDH curve.; output: Receives public X||Y point and secret scalar.

Return: True only after successful QSC generation

qsc::ecdh::try_generate_seeded_keypair

bool try_generate_seeded_keypair(curve selected, const secure_vector& seed, keypair& output) noexcept;

Purpose: Deterministically derive an ECDH curve keypair

Parameters: selected: Explicit ECDH curve.; seed: Secret seed with exact curve-specific length.; output: Generated public and private keys.

Return: True for valid seed and successful QSC generation

qsc::ecdh::try_public_from_private

bool try_public_from_private(curve selected, const secure_vector& private_key, byte_vector& public_key) noexcept;

Purpose: Validate scalar and compute canonical X||Y public point

Parameters: selected: Explicit ECDH curve.; private_key: Secret canonical scalar.; public_key: Output public point bytes.

Return: True if QSC returns success, including required blinding entropy

qsc::ecdh::try_key_exchange

bool try_key_exchange(curve selected, const secure_vector& private_key, const byte_vector& public_key, secure_vector& secret) noexcept;

Purpose: Perform validated nonzero ECDH agreement

Parameters: selected: Explicit ECDH curve.; private_key: Caller-owned secret scalar.; public_key: Peer's canonical X||Y point.; secret: Secure result, erased on any failure.

Return: True only for validated agreement and successful QSC operation

qsc::ecdh::generate_keypair

keypair generate_keypair(curve selected);

Purpose: Throwing convenience version of explicit keypair generation

Parameters: selected: Explicit ECDH curve.

Return: New keypair with non-exported-internal private scalar

qsc::ecdh::key_exchange

secure_vector key_exchange(curve selected, const secure_vector& private_key, const byte_vector& public_key);

Purpose: Throwing convenience version of explicit agreement

Parameters: selected: Explicit ECDH curve.; private_key: Local secret scalar.; public_key: Remote public point.

Return: Validated shared secret

qsc::ecdh::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::ecdh::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::ecdh::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::ecdh::shared_secret_size

std::size_t shared_secret_size() noexcept;

Purpose: Query shared secret size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::ecdh::seed_size

std::size_t seed_size() noexcept;

Purpose: Query seed size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::ecdh::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::ecdh::generate_seeded_keypair

keypair generate_seeded_keypair(const secure_vector& seed);

Purpose: Invoke generate seeded keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::ecdh::public_from_private

byte_vector public_from_private(const secure_vector& private_key);

Purpose: Public public from private operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::ecdh::key_exchange

secure_vector key_exchange(const secure_vector& private_key, const byte_vector& public_key);

Purpose: Public key exchange operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::ecdh::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::ecdh::try_generate_seeded_keypair

bool try_generate_seeded_keypair(const secure_vector& seed, keypair& output) noexcept;

Purpose: Non-throwing generate seeded keypair interface. Check the result before reading any output.

qsc::ecdh::try_public_from_private

bool try_public_from_private(const secure_vector& private_key, byte_vector& public_key) noexcept;

Purpose: Non-throwing public from private interface. Check the result before reading any output.

qsc::ecdh::try_key_exchange

bool try_key_exchange(const secure_vector& private_key, const byte_vector& public_key, secure_vector& shared_secret) noexcept;

Purpose: Non-throwing key exchange interface. Check the result before reading any output.

22.7 qsccpp_ecdsa.h — 26 callable declarations

Public ecdsa C++ functions and object methods.

qsc::ecdsa::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::ecdsa::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::ecdsa::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::ecdsa::signature_size

std::size_t signature_size() noexcept;

Purpose: Query signature size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::ecdsa::seed_size

std::size_t seed_size() noexcept;

Purpose: Query seed size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::ecdsa::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::ecdsa::generate_seeded_keypair

keypair generate_seeded_keypair(const secure_vector& seed);

Purpose: Invoke generate seeded keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::ecdsa::sign

byte_vector sign(const byte_vector& message, const secure_vector& private_key);

Purpose: Invoke the native QSC-backed sign operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::ecdsa::verify

byte_vector verify(const byte_vector& signed_message, const byte_vector& public_key);

Purpose: Invoke the native QSC-backed verify operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::ecdsa::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::ecdsa::try_generate_seeded_keypair

bool try_generate_seeded_keypair(const secure_vector& seed, keypair& output) noexcept;

Purpose: Non-throwing generate seeded keypair interface. Check the result before reading any output.

qsc::ecdsa::try_sign

bool try_sign(const byte_vector& message, const secure_vector& private_key, byte_vector& signed_message) noexcept;

Purpose: Non-throwing sign interface. Check the result before reading any output.

qsc::ecdsa::try_verify

bool try_verify(const byte_vector& signed_message, const byte_vector& public_key, byte_vector& message) noexcept;

Purpose: Non-throwing verify interface. Check the result before reading any output.

qsc::ecdsa::public_key_size

std::size_t public_key_size(curve selected) noexcept;

Purpose: Public verification-key length in bytes. Returns zero for invalid curve values

qsc::ecdsa::private_key_size

std::size_t private_key_size(curve selected) noexcept;

Purpose: Private signing-key length in bytes. Returns zero for invalid curve values

qsc::ecdsa::signature_size

std::size_t signature_size(curve selected) noexcept;

Purpose: Attached-signature prefix length in bytes. Returns zero for invalid curves

qsc::ecdsa::seed_size

std::size_t seed_size(curve selected) noexcept;

Purpose: Deterministic key-generation seed length in bytes. Returns zero for invalid curves

qsc::ecdsa::try_generate_keypair

bool try_generate_keypair(curve selected, keypair& output) noexcept;

Purpose: Generate a signing keypair with CSP entropy; securely clears output on error

Parameters: selected: NIST curve; output: Caller-owned keypair output.

Return: True on success. Not thread-safe for shared output

qsc::ecdsa::try_generate_seeded_keypair

bool try_generate_seeded_keypair(curve selected, const secure_vector& seed, keypair& output) noexcept;

Purpose: Generate a keypair from exactly seed_size(selected) private bytes

Parameters: selected: NIST curve; seed: Secret caller-owned seed; output: Keypair output.

Return: True on success; false on invalid size, RNG or allocation failure

qsc::ecdsa::try_sign

bool try_sign(curve selected, const byte_vector& message, const secure_vector& private_key, byte_vector& signed_message) noexcept;

Purpose: Create an attached signature using the curve-specific signer

Parameters: selected: NIST curve; message: Nonempty message; private_key: Native-format key.; signed_message: Attached output; cleared on failure. Uses expanded QSC private key.

Return: True if signing succeeded; false on any error. Thread-safe with independent inputs/outputs

qsc::ecdsa::try_sign_scalar

bool try_sign_scalar(curve selected, const byte_vector& message, const secure_vector& private_scalar, byte_vector& signed_message) noexcept;

Purpose: Sign with a raw, canonical curve scalar, NOT a generated expanded private key

Parameters: selected: Curve; message: Nonempty input; private_scalar: Raw big-endian scalar.; signed_message: Caller-owned attached output (cleared on failure).

Return: True only if QSC accepts scalar and returns complete attached signature

qsc::ecdsa::try_public_from_scalar

bool try_public_from_scalar(curve selected, const secure_vector& private_scalar, byte_vector& public_key) noexcept;

Purpose: Derive native public key from a raw canonical scalar

Parameters: selected: Curve; private_scalar: Native big-endian scalar of seed_size(selected) bytes.; public_key: Caller-owned public coordinates; cleared on failure.

Return: True only if the scalar is valid and QSC derives the point successfully

qsc::ecdsa::try_verify

bool try_verify(curve selected, const byte_vector& signed_message, const byte_vector& public_key, byte_vector& message) noexcept;

Purpose: Verify attached signature and recover message using a curve-specific public key

Parameters: selected: NIST curve; signed_message: Signature and message; public_key: Native-format public key.; message: Verified output, cleared on failure.

Return: True for a valid signature; false otherwise. Independent calls are thread-safe

qsc::ecdsa::generate_keypair

keypair generate_keypair(curve selected);

Purpose: Throwing keypair generation for explicit curve

Parameters: selected: NIST curve.

Return: Signing keypair. \throws crypto_exception on failure

qsc::ecdsa::sign

byte_vector sign(curve selected, const byte_vector& message, const secure_vector& private_key);

Purpose: Throwing attached signature creation

Parameters: selected: NIST curve; message: Message; private_key: Signing key.

Return: Signature followed by message. \throws crypto_exception on failure

qsc::ecdsa::verify

byte_vector verify(curve selected, const byte_vector& signed_message, const byte_vector& public_key);

Purpose: Throwing attached signature verification

Parameters: selected: NIST curve; signed_message: Signed data; public_key: Verification key.

Return: Verified message. \throws crypto_exception when verification fails

22.8 qsccpp_eddh.h — 23 callable declarations

Public eddh C++ functions and object methods.

qsc::eddh::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::eddh::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::eddh::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::eddh::shared_secret_size

std::size_t shared_secret_size() noexcept;

Purpose: Query shared secret size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::eddh::seed_size

std::size_t seed_size() noexcept;

Purpose: Query seed size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::eddh::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::eddh::generate_seeded_keypair

keypair generate_seeded_keypair(const secure_vector& seed);

Purpose: Invoke generate seeded keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::eddh::public_from_private

byte_vector public_from_private(const secure_vector& private_key);

Purpose: Public public from private operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::eddh::key_exchange

secure_vector key_exchange(const secure_vector& private_key, const byte_vector& public_key);

Purpose: Public key exchange operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::eddh::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::eddh::try_generate_seeded_keypair

bool try_generate_seeded_keypair(const secure_vector& seed, keypair& output) noexcept;

Purpose: Non-throwing generate seeded keypair interface. Check the result before reading any output.

qsc::eddh::try_public_from_private

bool try_public_from_private(const secure_vector& private_key, byte_vector& public_key) noexcept;

Purpose: Non-throwing public from private interface. Check the result before reading any output.

qsc::eddh::try_key_exchange

bool try_key_exchange(const secure_vector& private_key, const byte_vector& public_key, secure_vector& shared_secret) noexcept;

Purpose: Non-throwing key exchange interface. Check the result before reading any output.

qsc::eddh::public_key_size

std::size_t public_key_size(curve selected) noexcept;

Purpose: Public-key length in bytes (zero for invalid curve)

qsc::eddh::private_key_size

std::size_t private_key_size(curve selected) noexcept;

Purpose: Private-key length in bytes (zero for invalid curve)

qsc::eddh::shared_secret_size

std::size_t shared_secret_size(curve selected) noexcept;

Purpose: Shared-secret length in bytes (zero for invalid curve)

qsc::eddh::seed_size

std::size_t seed_size(curve selected) noexcept;

Purpose: Seed length for deterministic keypair generation (zero for invalid curve)

qsc::eddh::try_generate_keypair

bool try_generate_keypair(curve selected, keypair& output) noexcept;

Purpose: Generate a keypair using QSC CSP randomness

Parameters: selected: Curve; output: Caller-owned keypair.

Return: True on success; clears all output on failure

qsc::eddh::try_generate_seeded_keypair

bool try_generate_seeded_keypair(curve selected, const secure_vector& seed, keypair& output) noexcept;

Purpose: Generate deterministic keypair from exact-size seed

Parameters: selected: Curve; seed: Secret seed; output: Caller-owned keypair.

Return: True on success; false for invalid size/allocation failure

qsc::eddh::try_public_from_private

bool try_public_from_private(curve selected, const secure_vector& private_key, byte_vector& public_key) noexcept;

Purpose: Derive a public key from the curve-native private key

Parameters: selected: Curve; private_key: Secret scalar; public_key: Caller-owned output.

Return: True if valid; clears output on failure

qsc::eddh::try_key_exchange

bool try_key_exchange(curve selected, const secure_vector& private_key, const byte_vector& public_key, secure_vector& secret) noexcept;

Purpose: Compute checked X25519/X448 agreement with native low-order point rejection

Parameters: selected: Curve; private_key: Secret scalar; public_key: Peer public key; secret: Secure output.

Return: True only for an accepted nonzero shared secret; clears output on failure

qsc::eddh::generate_keypair

keypair generate_keypair(curve selected);

Purpose: Throwing explicit keypair generation

Parameters: selected: Curve.

Return: Generated keypair. \throws crypto_exception on failure

qsc::eddh::key_exchange

secure_vector key_exchange(curve selected, const secure_vector& private_key, const byte_vector& public_key);

Purpose: Throwing curve-specific key agreement

Parameters: selected: Curve; private_key: Private scalar; public_key: Peer point.

Return: Shared secret. \throws crypto_exception on failure

22.9 qsccpp_eddsa.h — 24 callable declarations

Public eddsa C++ functions and object methods.

qsc::eddsa::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::eddsa::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::eddsa::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::eddsa::signature_size

std::size_t signature_size() noexcept;

Purpose: Query signature size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::eddsa::seed_size

std::size_t seed_size() noexcept;

Purpose: Query seed size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::eddsa::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::eddsa::generate_seeded_keypair

keypair generate_seeded_keypair(const secure_vector& seed);

Purpose: Invoke generate seeded keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::eddsa::sign

byte_vector sign(const byte_vector& message, const secure_vector& private_key);

Purpose: Invoke the native QSC-backed sign operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::eddsa::verify

byte_vector verify(const byte_vector& signed_message, const byte_vector& public_key);

Purpose: Invoke the native QSC-backed verify operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::eddsa::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::eddsa::try_generate_seeded_keypair

bool try_generate_seeded_keypair(const secure_vector& seed, keypair& output) noexcept;

Purpose: Non-throwing generate seeded keypair interface. Check the result before reading any output.

qsc::eddsa::try_sign

bool try_sign(const byte_vector& message, const secure_vector& private_key, byte_vector& signed_message) noexcept;

Purpose: Non-throwing sign interface. Check the result before reading any output.

qsc::eddsa::try_verify

bool try_verify(const byte_vector& signed_message, const byte_vector& public_key, byte_vector& message) noexcept;

Purpose: Non-throwing verify interface. Check the result before reading any output.

qsc::eddsa::public_key_size

std::size_t public_key_size(curve selected) noexcept;

Purpose: Native public key length in bytes (zero for invalid curve)

qsc::eddsa::private_key_size

std::size_t private_key_size(curve selected) noexcept;

Purpose: Native secret key length in bytes (zero for invalid curve)

qsc::eddsa::signature_size

std::size_t signature_size(curve selected) noexcept;

Purpose: Attached signature length in bytes (zero for invalid curve)

qsc::eddsa::seed_size

std::size_t seed_size(curve selected) noexcept;

Purpose: Seed length in bytes (zero for invalid curve)

qsc::eddsa::try_generate_keypair

bool try_generate_keypair(curve selected, keypair& output) noexcept;

Purpose: Generate a keypair using QSC CSP; output is cleared on failure

Parameters: selected: Curve; output: Caller-owned generated keys.

Return: True on success. Do not share mutable output concurrently

qsc::eddsa::try_generate_seeded_keypair

bool try_generate_seeded_keypair(curve selected, const secure_vector& seed, keypair& output) noexcept;

Purpose: Derive a deterministic keypair from an exact-size secret seed

Parameters: selected: Curve; seed: Secret seed; output: Generated keys.

Return: True on success; false on invalid inputs or memory failure

qsc::eddsa::try_sign

bool try_sign(curve selected, const byte_vector& message, const secure_vector& private_key, byte_vector& signed_message) noexcept;

Purpose: Sign a nonempty message with curve-specific EdDSA

Parameters: selected: Curve; message: Input; private_key: Native private key; signed_message: Output.

Return: True on success; wipes and clears output on failure

qsc::eddsa::try_verify

bool try_verify(curve selected, const byte_vector& signed_message, const byte_vector& public_key, byte_vector& message) noexcept;

Purpose: Verify attached EdDSA signed message and recover plaintext

Parameters: selected: Curve; signed_message: Signed input; public_key: Native verification key; message: Output.

Return: True if valid. Clears output on invalid signature or failure

qsc::eddsa::generate_keypair

keypair generate_keypair(curve selected);

Purpose: Generate keypair or throw crypto_exception on failure

Parameters: selected: Curve.

Return: Generated keypair

qsc::eddsa::sign

byte_vector sign(curve selected, const byte_vector& message, const secure_vector& private_key);

Purpose: Sign and attach message or throw crypto_exception

Parameters: selected: Curve; message: Input; private_key: Secret key.

Return: Signed data

qsc::eddsa::verify

byte_vector verify(curve selected, const byte_vector& signed_message, const byte_vector& public_key);

Purpose: Verify and recover signed data, throwing on failure

Parameters: selected: Curve; signed_message: Signed data; public_key: Public key.

Return: Verified data

22.10 qsccpp_encoding.h — 8 callable declarations

Public encoding C++ functions and object methods.

qsc::encoding::hex_encode

std::string hex_encode(const byte_vector& data);

Purpose: Public hex encode operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::encoding::hex_decode

byte_vector hex_decode(const std::string& encoded);

Purpose: Public hex decode operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::encoding::try_hex_encode

bool try_hex_encode(const byte_vector& data, std::string& encoded) noexcept;

Purpose: Non-throwing hex encode interface. Check the result before reading any output.

qsc::encoding::try_hex_decode

bool try_hex_decode(const std::string& encoded, byte_vector& data) noexcept;

Purpose: Non-throwing hex decode interface. Check the result before reading any output.

qsc::encoding::base64_encode

std::string base64_encode(const byte_vector& data);

Purpose: Public base64 encode operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::encoding::base64_decode

byte_vector base64_decode(const std::string& encoded);

Purpose: Public base64 decode operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::encoding::try_base64_encode

bool try_base64_encode(const byte_vector& data, std::string& encoded) noexcept;

Purpose: Non-throwing base64 encode interface. Check the result before reading any output.

qsc::encoding::try_base64_decode

bool try_base64_decode(const std::string& encoded, byte_vector& data) noexcept;

Purpose: Non-throwing base64 decode interface. Check the result before reading any output.

22.11 qsccpp_entropy.h — 10 callable declarations

Public entropy C++ functions and object methods.

qsc::entropy::is_available

bool is_available(provider source) noexcept;

Purpose: Public is available operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::entropy::provider_to_string

const char* provider_to_string(provider source) noexcept;

Purpose: Public provider to string operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::entropy::try_generate

bool try_generate(provider source, std::size_t length, byte_vector& output) noexcept;

Purpose: Non-throwing generate interface. Check the result before reading any output.

qsc::entropy::generate

byte_vector generate(provider source, std::size_t length);

Purpose: Invoke the native QSC-backed generate operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::entropy::try_uint16

bool try_uint16(provider source, std::uint16_t& output) noexcept;

Purpose: Non-throwing uint16 interface. Check the result before reading any output.

qsc::entropy::try_uint32

bool try_uint32(provider source, std::uint32_t& output) noexcept;

Purpose: Non-throwing uint32 interface. Check the result before reading any output.

qsc::entropy::try_uint64

bool try_uint64(provider source, std::uint64_t& output) noexcept;

Purpose: Non-throwing uint64 interface. Check the result before reading any output.

qsc::entropy::external::initialize

bool initialize(qsc_etr_source_callback source, void* context) noexcept;

Purpose: Register a trusted external raw-entropy callback without taking ownership

Parameters: source: Native nonthrowing callback; partial reads must report written length.; context: Borrowed device or source state that outlives registration.

Return: False when callback is null or another source is already registered

qsc::entropy::external::try_generate

bool try_generate(std::size_t length, secure_vector& output) noexcept;

Purpose: Request SHAKE-512-conditioned ETR output with fail-closed semantics

Parameters: length: Requested bytes [1,QSC_ETR_SEED_MAX].; output: Secret output erased upon any source or allocation failure.

Return: True only when native source health/length checks pass

qsc::entropy::external::dispose

void dispose() noexcept;

Purpose: Unregister the process-global source, without freeing caller context

Details: Requires external synchronization against generate and initialize

22.12 qsccpp_exception.h — 6 callable declarations

Public exception C++ functions and object methods.

qsc::crypto_exception::crypto_exception

crypto_exception(const char* location, const char* origin, const char* message, error_code code);

Purpose: Constructs a crypto_exception instance; inspect subsequent readiness/status calls before use.

qsc::crypto_exception::what

const char* what() const noexcept override;

Purpose: Public what operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::crypto_exception::code

error_code code() const noexcept;

Purpose: Public code operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::crypto_exception::location

const std::string& location() const noexcept;

Purpose: Public location operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::crypto_exception::origin

const std::string& origin() const noexcept;

Purpose: Public origin operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::crypto_exception::message

const std::string& message() const noexcept;

Purpose: Public message operation; review the declared signature and its QSC-backed implementation for preconditions.

22.13 qsccpp_falcon.h — 10 callable declarations

Public falcon C++ functions and object methods.

qsc::falcon::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::falcon::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::falcon::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::falcon::signature_size

std::size_t signature_size() noexcept;

Purpose: Query signature size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::falcon::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::falcon::sign

byte_vector sign(const byte_vector& message, const secure_vector& private_key);

Purpose: Invoke the native QSC-backed sign operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::falcon::verify

byte_vector verify(const byte_vector& signed_message, const byte_vector& public_key);

Purpose: Invoke the native QSC-backed verify operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::falcon::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::falcon::try_sign

bool try_sign(const byte_vector& message, const secure_vector& private_key, byte_vector& signed_message) noexcept;

Purpose: Non-throwing sign interface. Check the result before reading any output.

qsc::falcon::try_verify

bool try_verify(const byte_vector& signed_message, const byte_vector& public_key, byte_vector& message) noexcept;

Purpose: Non-throwing verify interface. Check the result before reading any output.

22.14 qsccpp_hkdf.h — 12 callable declarations

Public hkdf C++ functions and object methods.

qsc::hkdf::expand256

byte_vector expand256(const byte_vector& key, const byte_vector& info, std::size_t output_length);

Purpose: Public expand256 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::hkdf::expand384

byte_vector expand384(const byte_vector& key, const byte_vector& info, std::size_t output_length);

Purpose: Public expand384 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::hkdf::expand512

byte_vector expand512(const byte_vector& key, const byte_vector& info, std::size_t output_length);

Purpose: Public expand512 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::hkdf::extract256

byte_vector extract256(const byte_vector& key, const byte_vector& salt, std::size_t output_length);

Purpose: Public extract256 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::hkdf::extract384

byte_vector extract384(const byte_vector& key, const byte_vector& salt, std::size_t output_length);

Purpose: Public extract384 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::hkdf::extract512

byte_vector extract512(const byte_vector& key, const byte_vector& salt, std::size_t output_length);

Purpose: Public extract512 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::hkdf::try_expand256

bool try_expand256(const byte_vector& key, const byte_vector& info, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing expand256 interface. Check the result before reading any output.

qsc::hkdf::try_expand384

bool try_expand384(const byte_vector& key, const byte_vector& info, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing expand384 interface. Check the result before reading any output.

qsc::hkdf::try_expand512

bool try_expand512(const byte_vector& key, const byte_vector& info, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing expand512 interface. Check the result before reading any output.

qsc::hkdf::try_extract256

bool try_extract256(const byte_vector& key, const byte_vector& salt, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing extract256 interface. Check the result before reading any output.

qsc::hkdf::try_extract384

bool try_extract384(const byte_vector& key, const byte_vector& salt, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing extract384 interface. Check the result before reading any output.

qsc::hkdf::try_extract512

bool try_extract512(const byte_vector& key, const byte_vector& salt, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing extract512 interface. Check the result before reading any output.

22.15 qsccpp_hmac.h — 9 callable declarations

Public hmac C++ functions and object methods.

qsc::hmac::compute256

byte_vector compute256(const byte_vector& key, const byte_vector& message);

Purpose: Invoke compute256 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::hmac::compute384

byte_vector compute384(const byte_vector& key, const byte_vector& message);

Purpose: Invoke compute384 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::hmac::compute512

byte_vector compute512(const byte_vector& key, const byte_vector& message);

Purpose: Invoke compute512 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::hmac::try_compute256

bool try_compute256(const byte_vector& key, const byte_vector& message, byte_vector& mac) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::hmac::try_compute384

bool try_compute384(const byte_vector& key, const byte_vector& message, byte_vector& mac) noexcept;

Purpose: Non-throwing compute384 interface. Check the result before reading any output.

qsc::hmac::try_compute512

bool try_compute512(const byte_vector& key, const byte_vector& message, byte_vector& mac) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

qsc::hmac::try_compute256

bool try_compute256(const byte* key, std::size_t key_length, const byte* message, std::size_t message_length, byte* mac, std::size_t mac_length) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::hmac::try_compute384

bool try_compute384(const byte* key, std::size_t key_length, const byte* message, std::size_t message_length, byte* mac, std::size_t mac_length) noexcept;

Purpose: Non-throwing compute384 interface. Check the result before reading any output.

qsc::hmac::try_compute512

bool try_compute512(const byte* key, std::size_t key_length, const byte* message, std::size_t message_length, byte* mac, std::size_t mac_length) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

22.16 qsccpp_hqc.h — 20 callable declarations

Public hqc C++ functions and object methods.

qsc::hqc::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::hqc::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::hqc::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::hqc::ciphertext_size

std::size_t ciphertext_size() noexcept;

Purpose: Query ciphertext size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::hqc::shared_secret_size

std::size_t shared_secret_size() noexcept;

Purpose: Query shared secret size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::hqc::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::hqc::encapsulate

encapsulation_result encapsulate(const byte_vector& public_key);

Purpose: Invoke the native QSC-backed encapsulate operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::hqc::decapsulate

secure_vector decapsulate(const byte_vector& ciphertext, const secure_vector& private_key);

Purpose: Invoke the native QSC-backed decapsulate operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::hqc::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::hqc::try_encapsulate

bool try_encapsulate(const byte_vector& public_key, encapsulation_result& output) noexcept;

Purpose: Non-throwing encapsulate interface. Check the result before reading any output.

qsc::hqc::try_decapsulate

bool try_decapsulate(const byte_vector& ciphertext, const secure_vector& private_key, secure_vector& shared_secret) noexcept;

Purpose: Non-throwing decapsulate interface. Check the result before reading any output.

qsc::hqc::public_key_size

std::size_t public_key_size(parameter_set algorithm) noexcept;

Purpose: Public-key length of the selected KEM (zero if invalid)

qsc::hqc::private_key_size

std::size_t private_key_size(parameter_set algorithm) noexcept;

Purpose: Private-key length of the selected KEM (zero if invalid)

qsc::hqc::ciphertext_size

std::size_t ciphertext_size(parameter_set algorithm) noexcept;

Purpose: Ciphertext length of the selected KEM (zero if invalid)

qsc::hqc::shared_secret_size

std::size_t shared_secret_size(parameter_set algorithm) noexcept;

Purpose: Shared-secret length of the selected KEM (zero if invalid)

qsc::hqc::try_generate_keypair

bool try_generate_keypair(parameter_set algorithm, keypair& output) noexcept;

Purpose: Generate a keypair with QSC CSP and exact-size outputs

Parameters: algorithm: Explicit native parameter set.; output: Caller-owned keypair.

Return: True on success. Output private key is erased on error

qsc::hqc::try_encapsulate

bool try_encapsulate(parameter_set algorithm, const byte_vector& public_key, encapsulation_result& output) noexcept;

Purpose: Encapsulate with explicit native public key

Parameters: algorithm: Explicit parameter set.; public_key: Exact-sized native public key.; output: Ciphertext and secure shared secret (both cleared on error).

Return: True only if QSC CSP and native encapsulation succeed

qsc::hqc::try_decapsulate

bool try_decapsulate(parameter_set algorithm, const byte_vector& ciphertext, const secure_vector& private_key, secure_vector& shared_secret) noexcept;

Purpose: Decapsulate ciphertext and securely own the resulting secret

Parameters: algorithm: Explicit parameter set.; ciphertext: Exact-sized native input.; private_key: Exact-sized native secret key.; shared_secret: Caller-owned result.

Return: Native QSC success status. Invalid ciphertext may produce a pseudorandom secret under implicit rejection; success does NOT establish ciphertext authenticity

qsc::hqc::try_generate_seeded_keypair

bool try_generate_seeded_keypair(parameter_set algorithm, const secure_vector& seed, keypair& output) noexcept;

Purpose: Derive a deterministic compact-key HQC pair from exactly 32 secret bytes

Parameters: algorithm: Explicit HQC level; seed: Caller-owned seed, never modified.; output: Keypair output, erased on error.

Return: True when QSC finishes key generation without memory failure

qsc::hqc::try_seeded_encapsulate

bool try_seeded_encapsulate(parameter_set algorithm, const secure_vector& seed, const byte_vector& public_key, encapsulation_result& output) noexcept;

Purpose: Deterministic encapsulation for controlled tests/interoperability only

Parameters: algorithm: Explicit HQC level; seed: Caller-owned 32-byte seed.; public_key: Exact-sized native key; output: Encapsulation result.

Return: True on valid sizes and successful allocation; do not reuse seeds in production

22.17 qsccpp_hwprovider.h — 25 callable declarations

Optional hardware provider and key custody operations; most entries are feature-guarded.

qsc::hardware::available

bool available() noexcept;

Purpose: Returns whether the QSC hardware API is available in this build

Return: True if QSCCPP was compiled with QSC_HARDWARE_ENABLED

qsc::hardware::default_key_policy

qsc_hw_key_policy default_key_policy(qsc_hw_algorithm algorithm, std::uint32_t usage) noexcept;

Purpose: Create a fail-closed hardware key custody policy

Parameters: algorithm: Required hardware key algorithm.; usage: Required qsc_hw_usage flag set.

Return: Size/version-qualified policy requiring hardware nonexportable custody

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::key

key() noexcept;

Purpose: Create an empty reference

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::~key

~key();

Purpose: Release a transient reference without deleting the persistent device key

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::key

key(key&& other) noexcept;

Purpose: Move the key reference; the backend handle remains stable

Parameters: other: Source reference, left empty.

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::operator=

key& operator=(key&& other) noexcept;

Purpose: Transfer ownership of a key reference

Parameters: other: Source reference, left empty.

Return: This object

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::valid

bool valid() const noexcept;

Purpose: Test whether a live reference is held

Return: True when the underlying key reference exists

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::clear

void clear() noexcept;

Purpose: Release an existing transient key and retained provider reference

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::get_info

qsc_hw_status get_info(qsc_hw_key_info& info) const noexcept;

Purpose: Obtain public key metadata without private material

Parameters: info: Size/version initialized output.

Return: Typed QSC status; invalid_parameter when no key is open

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::public_key

qsc_hw_status public_key(byte_vector& output, std::size_t& written) noexcept;

Purpose: Extract the public component into a caller-sized buffer

Parameters: output: Public-key output buffer; no allocation occurs.; written: In/out capacity and actual or required length.

Return: Typed status including buffer_too_small

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::sign

qsc_hw_status sign(const byte_vector& message, byte_vector& output, std::size_t& written) noexcept;

Purpose: Sign a message using the hardware key's enforced usage policy

Parameters: message: Caller-owned input bytes.; output: Caller-sized signature buffer, overwritten by QSC.; written: In/out capacity and actual or required byte count.

Return: Typed backend status. Does not fall back to a software signer

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::attest

qsc_hw_status attest(const byte_vector& challenge, byte_vector& output, std::size_t& written) noexcept;

Purpose: Request device attestation evidence without interpreting its trust chain

Parameters: challenge: Fresh challenge bytes from caller.; output: Caller-sized evidence buffer.; written: In/out buffer capacity and actual/required bytes.

Return: Typed status; evidence must be verified separately

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::key::agree

qsc_hw_status agree(const byte_vector& peer, secure_vector& output, std::size_t& written) noexcept;

Purpose: Derive an explicitly exportable secret using a protected private key

Parameters: peer: Public key bytes of the remote peer.; output: Secure caller-sized output. Backend enforces export policy.; written: In/out capacity and actual/required byte count.

Return: Typed QSC status; never exports private key material

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::provider

provider() noexcept;

Purpose: Create an empty provider wrapper

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::~provider

~provider();

Purpose: Release the last owner; actual close follows key lifetime

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::provider

provider(provider&& other) noexcept;

Purpose: Transfer provider state without copying backend handles

Parameters: other: Source provider, left empty.

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::operator=

provider& operator=(provider&& other) noexcept;

Purpose: Move-assign an owned provider

Parameters: other: Source provider, left empty.

Return: This provider

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::open

qsc_hw_status open(const qsc_hw_provider_ops& operations, void* backend_context) noexcept;

Purpose: Create a provider from a QSC backend operations descriptor

Parameters: operations: Versioned backend operations, copied by QSC.; backend_context: Caller-owned backend state; must remain alive through last key release.

Return: Typed status. Failed initialization transfers no ownership

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::close

qsc_hw_status close() noexcept;

Purpose: Explicitly close an idle provider

Return: Success when closed, busy while keys exist, otherwise a typed error

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::valid

bool valid() const noexcept;

Purpose: Test whether a live provider is owned

Return: True when the provider is open

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::capabilities

qsc_hw_status capabilities(qsc_hw_capabilities& capabilities) const noexcept;

Purpose: Obtain advertised backend capabilities, not certification claims

Parameters: capabilities: Size/version initialized output.

Return: Typed QSC status

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::generate_key

qsc_hw_status generate_key(const qsc_hw_key_policy& policy, key& output) noexcept;

Purpose: Generate a key that satisfies the requested custody and use policy

Parameters: policy: Caller-defined policy; use default_key_policy for strict defaults.; output: Newly opened transient reference; previous reference cleared.

Return: QSC status; no fallback on hardware failure

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::open_key

qsc_hw_status open_key(const std::string& locator, const qsc_hw_key_policy& policy, key& output) noexcept;

Purpose: Open a persistent key by locator under explicit custody requirements

Parameters: locator: Backend-specific persistent key identifier.; policy: Required algorithm, usage and custody.; output: Newly opened transient key reference; old reference cleared.

Return: Typed status, including policy denial

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::random

qsc_hw_status random(byte_vector& output) noexcept;

Purpose: Request device-provider random output; quality depends on backend

Parameters: output: Preallocated writable byte buffer.

Return: Typed QSC status

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

qsc::hardware::provider::invalidate

qsc_hw_status invalidate() noexcept;

Purpose: Invalidate current handles on backend reconnect/replacement

Return: Typed status; references still require release

Availability: Requires QSC_HARDWARE_ENABLED and a compatible QSC hardware-provider backend.

22.18 qsccpp_kmac.h — 12 callable declarations

Public kmac C++ functions and object methods.

qsc::kmac::compute128

byte_vector compute128(const byte_vector& key, const byte_vector& message, const byte_vector& customization, std::size_t output_length);

Purpose: Invoke compute128 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::kmac::compute256

byte_vector compute256(const byte_vector& key, const byte_vector& message, const byte_vector& customization, std::size_t output_length);

Purpose: Invoke compute256 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::kmac::compute512

byte_vector compute512(const byte_vector& key, const byte_vector& message, const byte_vector& customization, std::size_t output_length);

Purpose: Invoke compute512 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::kmac::xof128

byte_vector xof128(const byte_vector& key, const byte_vector& context, const byte_vector& label, std::size_t output_length);

Purpose: Public xof128 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::kmac::xof256

byte_vector xof256(const byte_vector& key, const byte_vector& context, const byte_vector& label, std::size_t output_length);

Purpose: Public xof256 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::kmac::xof512

byte_vector xof512(const byte_vector& key, const byte_vector& context, const byte_vector& label, std::size_t output_length);

Purpose: Public xof512 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::kmac::try_compute128

bool try_compute128(const byte_vector& key, const byte_vector& message, const byte_vector& customization, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing compute128 interface. Check the result before reading any output.

qsc::kmac::try_compute256

bool try_compute256(const byte_vector& key, const byte_vector& message, const byte_vector& customization, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::kmac::try_compute512

bool try_compute512(const byte_vector& key, const byte_vector& message, const byte_vector& customization, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

qsc::kmac::try_xof128

bool try_xof128(const byte_vector& key, const byte_vector& context, const byte_vector& label, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing xof128 interface. Check the result before reading any output.

qsc::kmac::try_xof256

bool try_xof256(const byte_vector& key, const byte_vector& context, const byte_vector& label, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing xof256 interface. Check the result before reading any output.

qsc::kmac::try_xof512

bool try_xof512(const byte_vector& key, const byte_vector& context, const byte_vector& label, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing xof512 interface. Check the result before reading any output.

22.19 qsccpp_mceliece.h — 18 callable declarations

Public mceliece C++ functions and object methods.

qsc::mceliece::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::mceliece::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mceliece::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mceliece::ciphertext_size

std::size_t ciphertext_size() noexcept;

Purpose: Query ciphertext size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mceliece::shared_secret_size

std::size_t shared_secret_size() noexcept;

Purpose: Query shared secret size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mceliece::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::mceliece::encapsulate

encapsulation_result encapsulate(const byte_vector& public_key);

Purpose: Invoke the native QSC-backed encapsulate operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::mceliece::decapsulate

secure_vector decapsulate(const byte_vector& ciphertext, const secure_vector& private_key);

Purpose: Invoke the native QSC-backed decapsulate operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::mceliece::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::mceliece::try_encapsulate

bool try_encapsulate(const byte_vector& public_key, encapsulation_result& output) noexcept;

Purpose: Non-throwing encapsulate interface. Check the result before reading any output.

qsc::mceliece::try_decapsulate

bool try_decapsulate(const byte_vector& ciphertext, const secure_vector& private_key, secure_vector& shared_secret) noexcept;

Purpose: Non-throwing decapsulate interface. Check the result before reading any output.

qsc::mceliece::public_key_size

std::size_t public_key_size(parameter_set algorithm) noexcept;

Purpose: Public-key length of the selected KEM (zero if invalid)

qsc::mceliece::private_key_size

std::size_t private_key_size(parameter_set algorithm) noexcept;

Purpose: Private-key length of the selected KEM (zero if invalid)

qsc::mceliece::ciphertext_size

std::size_t ciphertext_size(parameter_set algorithm) noexcept;

Purpose: Ciphertext length of the selected KEM (zero if invalid)

qsc::mceliece::shared_secret_size

std::size_t shared_secret_size(parameter_set algorithm) noexcept;

Purpose: Shared-secret length of the selected KEM (zero if invalid)

qsc::mceliece::try_generate_keypair

bool try_generate_keypair(parameter_set algorithm, keypair& output) noexcept;

Purpose: Generate a keypair with QSC CSP and exact-size outputs

Parameters: algorithm: Explicit native parameter set.; output: Caller-owned keypair.

Return: True on success. Output private key is erased on error

qsc::mceliece::try_encapsulate

bool try_encapsulate(parameter_set algorithm, const byte_vector& public_key, encapsulation_result& output) noexcept;

Purpose: Encapsulate with explicit native public key

Parameters: algorithm: Explicit parameter set.; public_key: Exact-sized native public key.; output: Ciphertext and secure shared secret (both cleared on error).

Return: True only if QSC CSP and native encapsulation succeed

qsc::mceliece::try_decapsulate

bool try_decapsulate(parameter_set algorithm, const byte_vector& ciphertext, const secure_vector& private_key, secure_vector& shared_secret) noexcept;

Purpose: Decapsulate ciphertext and securely own the resulting secret

Parameters: algorithm: Explicit parameter set.; ciphertext: Exact-sized native input.; private_key: Exact-sized native secret key.; shared_secret: Caller-owned result.

Return: Native QSC success status. Invalid ciphertext may produce a pseudorandom secret under implicit rejection; success does NOT establish ciphertext authenticity

22.20 qsccpp_mldsa.h — 20 callable declarations

Public mldsa C++ functions and object methods.

qsc::mldsa::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::mldsa::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mldsa::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mldsa::signature_size

std::size_t signature_size() noexcept;

Purpose: Query signature size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mldsa::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::mldsa::sign

byte_vector sign(const byte_vector& message, const secure_vector& private_key);

Purpose: Invoke the native QSC-backed sign operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::mldsa::verify

byte_vector verify(const byte_vector& signed_message, const byte_vector& public_key);

Purpose: Invoke the native QSC-backed verify operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::mldsa::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::mldsa::try_sign

bool try_sign(const byte_vector& message, const secure_vector& private_key, byte_vector& signed_message) noexcept;

Purpose: Non-throwing sign interface. Check the result before reading any output.

qsc::mldsa::try_verify

bool try_verify(const byte_vector& signed_message, const byte_vector& public_key, byte_vector& message) noexcept;

Purpose: Non-throwing verify interface. Check the result before reading any output.

qsc::mldsa::public_key_size

std::size_t public_key_size(parameter_set algorithm) noexcept;

Purpose: Obtain public-key length for a parameter set

Parameters: algorithm: Explicit ML-DSA parameter set.

Return: Byte length; zero if the parameter is invalid. Thread-safe

qsc::mldsa::private_key_size

std::size_t private_key_size(parameter_set algorithm) noexcept;

Purpose: Obtain private-key length for a parameter set

Parameters: algorithm: Explicit ML-DSA parameter set.

Return: Byte length; zero if the parameter is invalid. Thread-safe

qsc::mldsa::signature_size

std::size_t signature_size(parameter_set algorithm) noexcept;

Purpose: Obtain attached signature prefix length

Parameters: algorithm: Explicit ML-DSA parameter set.

Return: Signature length in bytes, or zero for invalid parameter. Thread-safe

qsc::mldsa::try_generate_keypair

bool try_generate_keypair(parameter_set algorithm, keypair& output) noexcept;

Purpose: Generate a new ML-DSA key pair using QSC CSP randomness

Parameters: algorithm: Explicit ML-DSA parameter set.; output: Receives public and secret key; cleared on error. Not shared across threads.

Return: True on success. False on allocation, RNG or parameter failure

qsc::mldsa::try_seeded_generate_keypair

bool try_seeded_generate_keypair(parameter_set algorithm, const secure_vector& seed, keypair& output) noexcept;

Purpose: Deterministically generate a key pair from a fixed-size seed

Parameters: algorithm: Explicit ML-DSA parameter set.; seed: Secret 32-byte seed; caller retains ownership and erases it.; output: Receives generated key pair; cleared on error.

Return: True on success; false on invalid input or allocation failure

qsc::mldsa::try_sign_ex

bool try_sign_ex(parameter_set algorithm, const byte_vector& message, const byte_vector& context, const secure_vector& private_key, byte_vector& signed_message) noexcept;

Purpose: Sign and append a message using an ML-DSA context

Parameters: algorithm: Explicit ML-DSA parameter set.; message: Message to sign, caller-owned.; context: Domain-separation context, at most 255 bytes; empty is allowed.; private_key: Secret signing key; its size must match algorithm.; signed_message: Signed output; cleared if signing fails.

Return: True on successful signing; false on input, RNG, or allocation failure

qsc::mldsa::try_verify_ex

bool try_verify_ex(parameter_set algorithm, const byte_vector& signed_message, const byte_vector& context, const byte_vector& public_key, byte_vector& message) noexcept;

Purpose: Verify an ML-DSA context-bound signed message and recover the message

Parameters: algorithm: Explicit ML-DSA parameter set.; signed_message: Signature followed by message, caller-owned.; context: Exact context used in signing, at most 255 bytes.; public_key: Public verification key; exact parameter-set length required.; message: Receives recovered message; securely cleared on failure.

Return: True on successful verification; false for invalid signatures or parameters

qsc::mldsa::generate_keypair

keypair generate_keypair(parameter_set algorithm);

Purpose: Generate an ML-DSA key pair, throwing on failure

Parameters: algorithm: Parameter set to use.

Return: Key pair with move-only private key. \throws crypto_exception on generation or allocation failure

qsc::mldsa::sign_ex

byte_vector sign_ex(parameter_set algorithm, const byte_vector& message, const byte_vector& context, const secure_vector& private_key);

Purpose: Sign an attached ML-DSA message with explicit context

Parameters: algorithm: Explicit parameter set.; message: Message input.; context: Domain separation context of at most 255 bytes.; private_key: Corresponding secret key.

Return: Attached signed message. \throws crypto_exception when signing fails

qsc::mldsa::verify_ex

byte_vector verify_ex(parameter_set algorithm, const byte_vector& signed_message, const byte_vector& context, const byte_vector& public_key);

Purpose: Verify and recover an attached ML-DSA signed message

Parameters: algorithm: Explicit parameter set.; signed_message: Signed message input.; context: Expected domain separation context.; public_key: Public verification key.

Return: Verified message. \throws crypto_exception if verification fails

22.21 qsccpp_mlkem.h — 21 callable declarations

Public mlkem C++ functions and object methods.

qsc::mlkem::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::mlkem::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mlkem::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mlkem::ciphertext_size

std::size_t ciphertext_size() noexcept;

Purpose: Query ciphertext size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mlkem::shared_secret_size

std::size_t shared_secret_size() noexcept;

Purpose: Query shared secret size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::mlkem::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::mlkem::encapsulate

encapsulation_result encapsulate(const byte_vector& public_key);

Purpose: Invoke the native QSC-backed encapsulate operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::mlkem::decapsulate

secure_vector decapsulate(const byte_vector& ciphertext, const secure_vector& private_key);

Purpose: Invoke the native QSC-backed decapsulate operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::mlkem::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::mlkem::try_encapsulate

bool try_encapsulate(const byte_vector& public_key, encapsulation_result& output) noexcept;

Purpose: Non-throwing encapsulate interface. Check the result before reading any output.

qsc::mlkem::try_decapsulate

bool try_decapsulate(const byte_vector& ciphertext, const secure_vector& private_key, secure_vector& shared_secret) noexcept;

Purpose: Decapsulates an ML-KEM ciphertext using FIPS 203 implicit rejection. The return value reports API/key validation and decapsulation completion. An invalid ciphertext still returns true when the private key is valid, with the FIPS 203 implicit-rejection shared secret written to shared_secret. Ciphertext validity must therefore be established by the consuming protocol's key confirmation/authentication. /

qsc::mlkem::public_key_size

std::size_t public_key_size(parameter_set algorithm) noexcept;

Purpose: Query the public key size

Parameters: algorithm: Explicit ML-KEM parameter set.

Return: Number of public-key bytes, or zero for an unsupported value

qsc::mlkem::private_key_size

std::size_t private_key_size(parameter_set algorithm) noexcept;

Purpose: Query the secret key size

Parameters: algorithm: Explicit ML-KEM parameter set.

Return: Number of secret-key bytes, or zero for an unsupported value

qsc::mlkem::ciphertext_size

std::size_t ciphertext_size(parameter_set algorithm) noexcept;

Purpose: Query ciphertext length

Parameters: algorithm: Explicit ML-KEM parameter set.

Return: Number of ciphertext bytes, or zero for an unsupported value

qsc::mlkem::shared_secret_size

std::size_t shared_secret_size(parameter_set algorithm) noexcept;

Purpose: Query shared-secret length

Parameters: algorithm: Explicit ML-KEM parameter set.

Return: Shared-secret length in bytes, or zero for an unsupported value

qsc::mlkem::try_generate_keypair

bool try_generate_keypair(parameter_set algorithm, keypair& output) noexcept;

Purpose: Generate an explicitly selected ML-KEM key pair using QSC CSP

Parameters: algorithm: Parameter set.; output: Result; secret key is move-only and erased on release.

Return: True on success. False on RNG, allocation, or parameter failure

qsc::mlkem::try_encapsulate

bool try_encapsulate(parameter_set algorithm, const byte_vector& public_key, encapsulation_result& output) noexcept;

Purpose: Encapsulate using a public key of the selected size

Parameters: algorithm: Parameter set.; public_key: Public encapsulation key.; output: Result; cleared on error.

Return: True on success. False for invalid key, RNG, or allocation error

qsc::mlkem::try_decapsulate

bool try_decapsulate(parameter_set algorithm, const byte_vector& ciphertext, const secure_vector& private_key, secure_vector& shared_secret) noexcept;

Purpose: Decapsulate with ML-KEM implicit rejection semantics

Parameters: algorithm: Parameter set.; ciphertext: Ciphertext of the exact parameter-set size.; private_key: Private decapsulation key; caller retains ownership.; shared_secret: Output; erased if the API returns false.

Return: True for a validated private key and completed decapsulation; invalid ciphertext is not reported separately

qsc::mlkem::generate_keypair

keypair generate_keypair(parameter_set algorithm);

Purpose: Generate an ML-KEM key pair, throwing on failure

Parameters: algorithm: Explicit parameter set.

Return: Generated key pair, owning secure private key. \throws crypto_exception when generation fails

qsc::mlkem::encapsulate

encapsulation_result encapsulate(parameter_set algorithm, const byte_vector& public_key);

Purpose: Encapsulate to an explicit ML-KEM public key

Parameters: algorithm: Explicit parameter set.; public_key: Public key input.

Return: Ciphertext and move-only secret. \throws crypto_exception when encapsulation fails

qsc::mlkem::decapsulate

secure_vector decapsulate(parameter_set algorithm, const byte_vector& ciphertext, const secure_vector& private_key);

Purpose: Decapsulate using an explicitly selected ML-KEM algorithm

Parameters: algorithm: Explicit parameter set.; ciphertext: Ciphertext input.; private_key: Private key input.

Return: Move-only derived secret; ciphertext integrity must be checked by a consuming protocol. \throws crypto_exception on API or secret-key validation failure

22.22 qsccpp_pbmac1.h — 4 callable declarations

Public pbmac1 C++ functions and object methods.

qsc::pbmac1::mac_size

std::size_t mac_size(digest algorithm) noexcept;

Purpose: Return MAC size, or zero on unsupported digest

Parameters: algorithm: Digest/HMAC choice.

Return: Digest size in bytes

qsc::pbmac1::try_compute

bool try_compute(digest algorithm, const secure_vector& password, const byte_vector& salt, std::uint64_t iterations, const byte_vector& message, byte_vector& output) noexcept;

Purpose: Calculate PBMAC1 authentication code with native PBKDF2

Parameters: algorithm: HMAC/PBKDF2 digest.; password: Borrowed secret, never cached.; salt: Public salt bytes.; iterations: PBKDF2 iterations [1,10000000].; message: Borrowed data.; output: MAC, cleared on failure.

Return: True only when native KDF/MAC succeeded; not thread-safe for shared outputs

qsc::pbmac1::try_verify

bool try_verify(digest algorithm, const secure_vector& password, const byte_vector& salt, std::uint64_t iterations, const byte_vector& message, const byte_vector& expected) noexcept;

Purpose: Constant-time verify using native PBMAC1 verifier

Parameters: algorithm: Digest choice.; password: Borrowed secret.; salt: Public salt.; iterations: PBKDF2 count.; message: Message data.; expected: Exact-length code.

Return: False on mismatch or invalid parameters; does not reveal which

qsc::pbmac1::try_derive_key

bool try_derive_key(digest algorithm, const secure_vector& password, const byte_vector& salt, std::uint64_t iterations, std::size_t length, secure_vector& output) noexcept;

Purpose: Derive a bounded PBMAC1 key into securely owned memory

Parameters: algorithm: Digest.; password: Borrowed secret.; salt: Public salt.; iterations: PBKDF2 count.; length: Desired key size [1,64].; output: Secret key, erased on failure.

Return: True for successful native PBKDF2

22.23 qsccpp_qmac.h — 2 callable declarations

Public qmac C++ functions and object methods.

qsc::qmac::compute

byte_vector compute(const byte_vector& key, const byte_vector& nonce, const byte_vector& info, const byte_vector& message);

Purpose: Invoke the native QSC-backed compute operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::qmac::try_compute

bool try_compute(const byte_vector& key, const byte_vector& nonce, const byte_vector& info, const byte_vector& message, byte_vector& mac) noexcept;

Purpose: Non-throwing compute interface. Check the result before reading any output.

22.24 qsccpp_rcs.h — 8 callable declarations

Public rcs C++ functions and object methods.

qsc::rcs::encrypt256

byte_vector encrypt256(const rcs256_key& key, const rcs_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, const byte_vector& info);

Purpose: Invoke encrypt256 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::rcs::decrypt256

byte_vector decrypt256(const rcs256_key& key, const rcs_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, const byte_vector& info);

Purpose: Invoke decrypt256 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::rcs::encrypt512

byte_vector encrypt512(const rcs512_key& key, const rcs_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, const byte_vector& info);

Purpose: Invoke encrypt512 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::rcs::decrypt512

byte_vector decrypt512(const rcs512_key& key, const rcs_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, const byte_vector& info);

Purpose: Invoke decrypt512 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::rcs::try_encrypt256

bool try_encrypt256(const rcs256_key& key, const rcs_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, const byte_vector& info, byte_vector& ciphertext) noexcept;

Purpose: Non-throwing encrypt256 interface. Check the result before reading any output.

qsc::rcs::try_decrypt256

bool try_decrypt256(const rcs256_key& key, const rcs_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, const byte_vector& info, byte_vector& plaintext) noexcept;

Purpose: Non-throwing decrypt256 interface. Check the result before reading any output.

qsc::rcs::try_encrypt512

bool try_encrypt512(const rcs512_key& key, const rcs_nonce& nonce, const byte_vector& plaintext, const byte_vector& associated_data, const byte_vector& info, byte_vector& ciphertext) noexcept;

Purpose: Non-throwing encrypt512 interface. Check the result before reading any output.

qsc::rcs::try_decrypt512

bool try_decrypt512(const rcs512_key& key, const rcs_nonce& nonce, const byte_vector& ciphertext, const byte_vector& associated_data, const byte_vector& info, byte_vector& plaintext) noexcept;

Purpose: Non-throwing decrypt512 interface. Check the result before reading any output.

22.25 qsccpp_scb.h — 4 callable declarations

Public scb C++ functions and object methods.

qsc::scb::generate256

byte_vector generate256(const scb256_seed& seed, const byte_vector& info, std::size_t output_length, std::size_t cpu_cost, std::size_t memory_cost);

Purpose: Invoke generate256 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::scb::generate512

byte_vector generate512(const scb512_seed& seed, const byte_vector& info, std::size_t output_length, std::size_t cpu_cost, std::size_t memory_cost);

Purpose: Invoke generate512 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::scb::try_generate256

bool try_generate256(const scb256_seed& seed, const byte_vector& info, std::size_t output_length, std::size_t cpu_cost, std::size_t memory_cost, byte_vector& output) noexcept;

Purpose: Non-throwing generate256 interface. Check the result before reading any output.

qsc::scb::try_generate512

bool try_generate512(const scb512_seed& seed, const byte_vector& info, std::size_t output_length, std::size_t cpu_cost, std::size_t memory_cost, byte_vector& output) noexcept;

Purpose: Non-throwing generate512 interface. Check the result before reading any output.

22.26 qsccpp_secrand.h — 7 callable declarations

Public secrand C++ functions and object methods.

qsc::secrand::try_initialize

bool try_initialize(const secure_vector& seed, const byte_vector& customization) noexcept;

Purpose: Non-throwing initialize interface. Check the result before reading any output.

qsc::secrand::initialize

void initialize(const secure_vector& seed, const byte_vector& customization);

Purpose: Public initialize operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::secrand::dispose

void dispose() noexcept;

Purpose: Public dispose operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::secrand::try_generate

bool try_generate(std::size_t length, byte_vector& output) noexcept;

Purpose: Non-throwing generate interface. Check the result before reading any output.

qsc::secrand::generate

byte_vector generate(std::size_t length);

Purpose: Invoke the native QSC-backed generate operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::secrand::next_uint32

std::uint32_t next_uint32() noexcept;

Purpose: Public next uint32 operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::secrand::next_uint64

std::uint64_t next_uint64() noexcept;

Purpose: Public next uint64 operation; review the declared signature and its QSC-backed implementation for preconditions.

22.27 qsccpp_secure.h — 16 callable declarations

Secure allocator, move-only secret vector and allocator interoperability.

qsc::secure_byte_allocator::std::allocator<T>

template <class T, typename std::enable_if<!std::is_same<T, value_type>::value, int>::type = 0> operator std::allocator<T>() const noexcept;

Purpose: Convert this stateless allocator for standard-library metadata

Details: MSVC constructs its vector container-proxy allocator from the vector's byte allocator. The rebound standard allocator must therefore be constructible from this allocator instance. This conversion is for non-byte bookkeeping types only; storage for secure_vector's byte elements still uses secure_byte_allocator and is securely erased before deallocation

Return: Stateless standard allocator for T, without ownership transfer

qsc::secure_byte_allocator::allocate

byte* allocate(std::size_t n);

Purpose: Allocate storage for n bytes

Parameters: n: Number of bytes to allocate.

Return: Allocated storage, or throws std::bad_alloc on failure

qsc::secure_byte_allocator::deallocate

void deallocate(byte* pointer, std::size_t n) noexcept;

Purpose: Erase and release n bytes of allocated storage

Parameters: pointer: Storage obtained from allocate.; n: Allocation length in bytes.

qsc::operator==

inline bool operator==(const secure_byte_allocator& left, const secure_byte_allocator& right) noexcept;

Purpose: Compare interchangeable secure allocators

Parameters: left: First allocator.; right: Second allocator.

Return: true

qsc::operator!=

inline bool operator!=(const secure_byte_allocator& left, const secure_byte_allocator& right) noexcept;

Purpose: Compare interchangeable secure allocators

Parameters: left: First allocator.; right: Second allocator.

Return: false

qsc::secure_vector::secure_vector

secure_vector();

Purpose: Constructs a secure_vector instance; inspect subsequent readiness/status calls before use.

qsc::secure_vector::secure_vector

explicit secure_vector(std::size_t length);

Purpose: Constructs a secure_vector instance; inspect subsequent readiness/status calls before use.

qsc::secure_vector::~secure_vector

~secure_vector();

Purpose: Destructor for secure_vector; releases native resources according to the owner lifecycle.

qsc::secure_vector::secure_vector

secure_vector(secure_vector&& other) noexcept;

Purpose: Constructs a secure_vector instance; inspect subsequent readiness/status calls before use.

qsc::secure_vector::operator=

secure_vector& operator=(secure_vector&& other) noexcept;

Purpose: C++ operator defined by the declaration. Its semantics and ownership are governed by the operand types.

qsc::secure_vector::data

byte* data() noexcept;

Purpose: Public data operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::secure_vector::data

const byte* data() const noexcept;

Purpose: Public data operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::secure_vector::size

std::size_t size() const noexcept;

Purpose: Public size operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::secure_vector::empty

bool empty() const noexcept;

Purpose: Public empty operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::secure_vector::resize

void resize(std::size_t length);

Purpose: Public resize operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::secure_vector::clear

void clear() noexcept;

Purpose: Public clear operation; review the declared signature and its QSC-backed implementation for preconditions.

22.28 qsccpp_sha2.h — 33 callable declarations

One-shot and streaming SHA-2 interfaces.

qsc::sha2::stream256::stream256

stream256() noexcept;

Purpose: Initialize a new digest state

qsc::sha2::stream256::~stream256

~stream256();

Purpose: Erase a live state on destruction

qsc::sha2::stream256::reset

void reset() noexcept;

Purpose: Restart the digest and discard the previous state

qsc::sha2::stream256::try_update

bool try_update(const byte* message, std::size_t length) noexcept;

Purpose: Absorb a caller-owned message fragment

Parameters: message: Pointer to bytes, or null only if length is zero.; length: Number of input bytes.

Return: False for invalid input or a finalized state

qsc::sha2::stream256::try_update

bool try_update(const byte_vector& message) noexcept;

Purpose: Absorb a caller-owned vector

Parameters: message: Input fragment.

Return: False if the state was previously finalized

qsc::sha2::stream256::try_finalize

bool try_finalize(byte_vector& output) noexcept;

Purpose: Finalize the digest and erase native state

Parameters: output: Receives exactly 32 bytes on success.

Return: False if already finalized or memory allocation fails

qsc::sha2::stream256::finalize

byte_vector finalize();

Purpose: Finalize or throw crypto_exception on failure

Return: Exactly 32 bytes

qsc::sha2::stream256::active

bool active() const noexcept;

Purpose: Whether new message blocks can be absorbed

Return: True until successful finalization

qsc::sha2::stream384::stream384

stream384() noexcept;

Purpose: Initialize a new digest state

qsc::sha2::stream384::~stream384

~stream384();

Purpose: Erase a live state on destruction

qsc::sha2::stream384::reset

void reset() noexcept;

Purpose: Restart the digest and discard the previous state

qsc::sha2::stream384::try_update

bool try_update(const byte* message, std::size_t length) noexcept;

Purpose: Absorb a caller-owned message fragment

Parameters: message: Pointer to bytes, or null only if length is zero.; length: Number of input bytes.

Return: False for invalid input or a finalized state

qsc::sha2::stream384::try_update

bool try_update(const byte_vector& message) noexcept;

Purpose: Absorb a caller-owned vector

Parameters: message: Input fragment.

Return: False if the state was previously finalized

qsc::sha2::stream384::try_finalize

bool try_finalize(byte_vector& output) noexcept;

Purpose: Finalize the digest and erase native state

Parameters: output: Receives exactly 48 bytes on success.

Return: False if already finalized or memory allocation fails

qsc::sha2::stream384::finalize

byte_vector finalize();

Purpose: Finalize or throw crypto_exception on failure

Return: Exactly 48 bytes

qsc::sha2::stream384::active

bool active() const noexcept;

Purpose: Whether new message blocks can be absorbed

Return: True until successful finalization

qsc::sha2::stream512::stream512

stream512() noexcept;

Purpose: Initialize a new digest state

qsc::sha2::stream512::~stream512

~stream512();

Purpose: Erase a live state on destruction

qsc::sha2::stream512::reset

void reset() noexcept;

Purpose: Restart the digest and discard the previous state

qsc::sha2::stream512::try_update

bool try_update(const byte* message, std::size_t length) noexcept;

Purpose: Absorb a caller-owned message fragment

Parameters: message: Pointer to bytes, or null only if length is zero.; length: Number of input bytes.

Return: False for invalid input or a finalized state

qsc::sha2::stream512::try_update

bool try_update(const byte_vector& message) noexcept;

Purpose: Absorb a caller-owned vector

Parameters: message: Input fragment.

Return: False if the state was previously finalized

qsc::sha2::stream512::try_finalize

bool try_finalize(byte_vector& output) noexcept;

Purpose: Finalize the digest and erase native state

Parameters: output: Receives exactly 64 bytes on success.

Return: False if already finalized or memory allocation fails

qsc::sha2::stream512::finalize

byte_vector finalize();

Purpose: Finalize or throw crypto_exception on failure

Return: Exactly 64 bytes

qsc::sha2::stream512::active

bool active() const noexcept;

Purpose: Whether new message blocks can be absorbed

Return: True until successful finalization

qsc::sha2::compute256

byte_vector compute256(const byte_vector& message);

Purpose: Invoke compute256 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::sha2::compute384

byte_vector compute384(const byte_vector& message);

Purpose: Invoke compute384 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::sha2::compute512

byte_vector compute512(const byte_vector& message);

Purpose: Invoke compute512 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::sha2::try_compute256

bool try_compute256(const byte_vector& message, byte_vector& digest) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::sha2::try_compute384

bool try_compute384(const byte_vector& message, byte_vector& digest) noexcept;

Purpose: Non-throwing compute384 interface. Check the result before reading any output.

qsc::sha2::try_compute512

bool try_compute512(const byte_vector& message, byte_vector& digest) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

qsc::sha2::try_compute256

bool try_compute256(const byte* message, std::size_t message_length, byte* digest, std::size_t digest_length) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::sha2::try_compute384

bool try_compute384(const byte* message, std::size_t message_length, byte* digest, std::size_t digest_length) noexcept;

Purpose: Non-throwing compute384 interface. Check the result before reading any output.

qsc::sha2::try_compute512

bool try_compute512(const byte* message, std::size_t message_length, byte* digest, std::size_t digest_length) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

22.29 qsccpp_sha3.h — 33 callable declarations

One-shot and streaming SHA-3 interfaces.

qsc::sha3::stream128::stream128

stream128() noexcept;

Purpose: Initialize an empty SHA3 state

qsc::sha3::stream128::~stream128

~stream128();

Purpose: Erase a still-active native state

qsc::sha3::stream128::reset

void reset() noexcept;

Purpose: Reset to a fresh digest state

qsc::sha3::stream128::try_update

bool try_update(const byte* message, std::size_t length) noexcept;

Purpose: Absorb caller-owned input

Parameters: message: Bytes, or null only if length is zero.; length: Input byte count.

Return: False on finalized state or invalid input

qsc::sha3::stream128::try_update

bool try_update(const byte_vector& message) noexcept;

Purpose: Absorb caller-owned vector contents

Parameters: message: Input fragment.

Return: False after finalization

qsc::sha3::stream128::try_finalize

bool try_finalize(byte_vector& output) noexcept;

Purpose: Produce digest and erase native state

Parameters: output: Receives exactly 16 digest bytes.

Return: False if finalized or output allocation fails

qsc::sha3::stream128::finalize

byte_vector finalize();

Purpose: Finalize, throwing on failure

Return: Digest bytes

qsc::sha3::stream128::active

bool active() const noexcept;

Purpose: Check whether updates remain permitted

Return: True until successful finalization

qsc::sha3::stream256::stream256

stream256() noexcept;

Purpose: Initialize an empty SHA3 state

qsc::sha3::stream256::~stream256

~stream256();

Purpose: Erase a still-active native state

qsc::sha3::stream256::reset

void reset() noexcept;

Purpose: Reset to a fresh digest state

qsc::sha3::stream256::try_update

bool try_update(const byte* message, std::size_t length) noexcept;

Purpose: Absorb caller-owned input

Parameters: message: Bytes, or null only if length is zero.; length: Input byte count.

Return: False on finalized state or invalid input

qsc::sha3::stream256::try_update

bool try_update(const byte_vector& message) noexcept;

Purpose: Absorb caller-owned vector contents

Parameters: message: Input fragment.

Return: False after finalization

qsc::sha3::stream256::try_finalize

bool try_finalize(byte_vector& output) noexcept;

Purpose: Produce digest and erase native state

Parameters: output: Receives exactly 32 digest bytes.

Return: False if finalized or output allocation fails

qsc::sha3::stream256::finalize

byte_vector finalize();

Purpose: Finalize, throwing on failure

Return: Digest bytes

qsc::sha3::stream256::active

bool active() const noexcept;

Purpose: Check whether updates remain permitted

Return: True until successful finalization

qsc::sha3::stream512::stream512

stream512() noexcept;

Purpose: Initialize an empty SHA3 state

qsc::sha3::stream512::~stream512

~stream512();

Purpose: Erase a still-active native state

qsc::sha3::stream512::reset

void reset() noexcept;

Purpose: Reset to a fresh digest state

qsc::sha3::stream512::try_update

bool try_update(const byte* message, std::size_t length) noexcept;

Purpose: Absorb caller-owned input

Parameters: message: Bytes, or null only if length is zero.; length: Input byte count.

Return: False on finalized state or invalid input

qsc::sha3::stream512::try_update

bool try_update(const byte_vector& message) noexcept;

Purpose: Absorb caller-owned vector contents

Parameters: message: Input fragment.

Return: False after finalization

qsc::sha3::stream512::try_finalize

bool try_finalize(byte_vector& output) noexcept;

Purpose: Produce digest and erase native state

Parameters: output: Receives exactly 64 digest bytes.

Return: False if finalized or output allocation fails

qsc::sha3::stream512::finalize

byte_vector finalize();

Purpose: Finalize, throwing on failure

Return: Digest bytes

qsc::sha3::stream512::active

bool active() const noexcept;

Purpose: Check whether updates remain permitted

Return: True until successful finalization

qsc::sha3::compute128

byte_vector compute128(const byte_vector& message);

Purpose: Invoke compute128 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::sha3::compute256

byte_vector compute256(const byte_vector& message);

Purpose: Invoke compute256 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::sha3::compute512

byte_vector compute512(const byte_vector& message);

Purpose: Invoke compute512 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::sha3::try_compute128

bool try_compute128(const byte_vector& message, byte_vector& digest) noexcept;

Purpose: Non-throwing compute128 interface. Check the result before reading any output.

qsc::sha3::try_compute256

bool try_compute256(const byte_vector& message, byte_vector& digest) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::sha3::try_compute512

bool try_compute512(const byte_vector& message, byte_vector& digest) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

qsc::sha3::try_compute128

bool try_compute128(const byte* message, std::size_t message_length, byte* digest, std::size_t digest_length) noexcept;

Purpose: Non-throwing compute128 interface. Check the result before reading any output.

qsc::sha3::try_compute256

bool try_compute256(const byte* message, std::size_t message_length, byte* digest, std::size_t digest_length) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::sha3::try_compute512

bool try_compute512(const byte* message, std::size_t message_length, byte* digest, std::size_t digest_length) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

22.30 qsccpp_shake.h — 20 callable declarations

Public shake C++ functions and object methods.

qsc::shake::compute128

byte_vector compute128(const byte_vector& key, std::size_t output_length);

Purpose: Invoke compute128 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::shake::compute256

byte_vector compute256(const byte_vector& key, std::size_t output_length);

Purpose: Invoke compute256 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::shake::compute512

byte_vector compute512(const byte_vector& key, std::size_t output_length);

Purpose: Invoke compute512 through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::shake::try_compute128

bool try_compute128(const byte_vector& key, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing compute128 interface. Check the result before reading any output.

qsc::shake::try_compute256

bool try_compute256(const byte_vector& key, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::shake::try_compute512

bool try_compute512(const byte_vector& key, std::size_t output_length, byte_vector& output) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

qsc::shake::try_compute128

bool try_compute128(const byte* key, std::size_t key_length, byte* output, std::size_t output_length) noexcept;

Purpose: Non-throwing compute128 interface. Check the result before reading any output.

qsc::shake::try_compute256

bool try_compute256(const byte* key, std::size_t key_length, byte* output, std::size_t output_length) noexcept;

Purpose: Non-throwing compute256 interface. Check the result before reading any output.

qsc::shake::try_compute512

bool try_compute512(const byte* key, std::size_t key_length, byte* output, std::size_t output_length) noexcept;

Purpose: Non-throwing compute512 interface. Check the result before reading any output.

qsc::shake::xof_state::xof_state

explicit xof_state(strength selected) noexcept;

Purpose: Construct initialized XOF state

Parameters: selected: Desired SHAKE instance.

qsc::shake::xof_state::~xof_state

~xof_state();

Purpose: Dispose and erase native state; no exception thrown

qsc::shake::xof_state::reset

void reset() noexcept;

Purpose: Reinitialize XOF state, discarding prior input and output position

qsc::shake::xof_state::try_update

bool try_update(const byte* data, std::size_t length) noexcept;

Purpose: Absorb additional data before finalization

Parameters: data: Bytes to absorb (may be null only when length is zero).; length: Byte count.

Return: False if already finalized or pointer invalid

qsc::shake::xof_state::try_update

bool try_update(const byte_vector& data) noexcept;

Purpose: Absorb byte-vector input before finalization

Parameters: data: Caller-owned bytes.

Return: False if already finalized

qsc::shake::xof_state::try_finalize

bool try_finalize() noexcept;

Purpose: Finalize the input sponge before squeezing

Return: False if called after finalization. Reinitialize for a new message

qsc::shake::xof_state::try_squeeze

bool try_squeeze(byte* output, std::size_t length) noexcept;

Purpose: Generate any number of output bytes in multiple calls

Parameters: output: Caller-owned output, nonnull for nonzero length.; length: Byte count to generate.

Return: False before finalization

qsc::shake::xof_state::try_squeeze

bool try_squeeze(std::size_t length, byte_vector& output) noexcept;

Purpose: Generate XOF output into vector (cleared on error)

Parameters: length: Desired output bytes; output: Caller-owned result.

Return: True if finalized and allocation succeeds

qsc::shake::try_cshake128

bool try_cshake128(const byte_vector& key, const byte_vector& name, const byte_vector& custom, std::size_t output_length, byte_vector& output) noexcept;

Purpose: cSHAKE-128 one-shot named/customized XOF

Parameters: key: Input bytes; name: Function-name domain; custom: Customization string.; output_length: Desired bytes; output: Caller-owned result, cleared on error.

Return: True for valid lengths and successful output allocation

qsc::shake::try_cshake256

bool try_cshake256(const byte_vector& key, const byte_vector& name, const byte_vector& custom, std::size_t output_length, byte_vector& output) noexcept;

Purpose: cSHAKE-256 one-shot named/customized XOF

Parameters: key: Input; name: Function name; custom: Custom string.; output_length: Requested bytes; output: Result.

Return: True on success. Independent invocations are thread-safe

qsc::shake::try_cshake512

bool try_cshake512(const byte_vector& key, const byte_vector& name, const byte_vector& custom, std::size_t output_length, byte_vector& output) noexcept;

Purpose: cSHAKE-512 one-shot QSC-extended named/customized XOF

Parameters: key: Input; name: Function name; custom: Custom string.; output_length: Requested bytes; output: Result.

Return: True on success. Independent invocations are thread-safe

22.31 qsccpp_slhdsa.h — 10 callable declarations

Public slhdsa C++ functions and object methods.

qsc::slhdsa::public_key_size

std::size_t public_key_size(parameter_set algorithm) noexcept;

Purpose: Get public-key size for parameter set; zero for invalid values

qsc::slhdsa::private_key_size

std::size_t private_key_size(parameter_set algorithm) noexcept;

Purpose: Get private-key size for parameter set; zero for invalid values

qsc::slhdsa::signature_size

std::size_t signature_size(parameter_set algorithm) noexcept;

Purpose: Get detached signature size for parameter set; zero for invalid values

qsc::slhdsa::seed_size

std::size_t seed_size(parameter_set algorithm) noexcept;

Purpose: Get deterministic key-generation seed size; zero for invalid values

qsc::slhdsa::try_generate_keypair

bool try_generate_keypair(parameter_set algorithm, keypair& output) noexcept;

Purpose: Generate independent SLH-DSA keypair using CSP entropy

Parameters: algorithm: Explicit FIPS 205 variant.; output: Keypair output, cleared on error.

Return: True for successful generation. Caller must serialize shared-output access

qsc::slhdsa::try_seeded_generate_keypair

bool try_seeded_generate_keypair(parameter_set algorithm, const secure_vector& seed, keypair& output) noexcept;

Purpose: Generate deterministic keypair from an exact-length secret seed

Parameters: algorithm: Explicit variant.; seed: Caller-owned secret seed.; output: Keypair output, securely cleared on error.

Return: True on success, false for invalid inputs or memory failure

qsc::slhdsa::try_sign

bool try_sign(parameter_set algorithm, const byte_vector& message, const byte_vector& context, const secure_vector& private_key, byte_vector& signature, bool randomized = false) noexcept;

Purpose: Sign a message with a detached FIPS 205 signature

Parameters: algorithm: Variant.; message: Nonempty input bytes.; context: FIPS 205 domain context, at most 255 bytes.; private_key: Native secret signing key of exact variant size.; signature: Detached signature, overwritten or securely cleared on failure.; randomized: If true, sample per-message signing randomness through CSP.

Return: True on success; failed RNG, allocation and invalid inputs return false

qsc::slhdsa::try_verify

bool try_verify(parameter_set algorithm, const byte_vector& signature, const byte_vector& message, const byte_vector& context, const byte_vector& public_key) noexcept;

Purpose: Verify a detached FIPS 205 signature

Parameters: algorithm: Variant.; signature: Signature bytes of exact size.; message: Nonempty caller-owned data.; context: Domain context.; public_key: Exact-sized public verification key.

Return: True for a valid signature, otherwise false. Does not modify inputs

qsc::slhdsa::try_sign_prehash

bool try_sign_prehash(parameter_set algorithm, prehash hash, const byte_vector& message, const byte_vector& context, const secure_vector& private_key, byte_vector& signature, bool randomized = false) noexcept;

Purpose: Sign a message with the selected FIPS 205 prehash algorithm

Parameters: algorithm: Variant; hash: Valid SHA-2/SHAKE prehash selector.; message: Nonempty input bytes; context: Optional domain context.; private_key: Native signing key; signature: Detached output.; randomized: If true, use QSC CSP for signing randomness.

Return: True on success, false on any input, RNG or allocation failure

qsc::slhdsa::try_verify_prehash

bool try_verify_prehash(parameter_set algorithm, prehash hash, const byte_vector& signature, const byte_vector& message, const byte_vector& context, const byte_vector& public_key) noexcept;

Purpose: Verify detached prehash signature using selected FIPS 205 parameters

Parameters: algorithm: Variant; hash: Valid prehash selector.; signature: Detached signature; message: Nonempty signed message.; context: Domain context; public_key: Exact-size public key.

Return: True if signature verifies, otherwise false

22.32 qsccpp_sphincsplus.h — 10 callable declarations

Public sphincsplus C++ functions and object methods.

qsc::sphincsplus::algorithm_name

const char* algorithm_name() noexcept;

Purpose: Public algorithm name operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::sphincsplus::public_key_size

std::size_t public_key_size() noexcept;

Purpose: Query public key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::sphincsplus::private_key_size

std::size_t private_key_size() noexcept;

Purpose: Query private key size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::sphincsplus::signature_size

std::size_t signature_size() noexcept;

Purpose: Query signature size metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::sphincsplus::generate_keypair

keypair generate_keypair();

Purpose: Invoke generate keypair through the QSC-backed wrapper. Check relevant output lengths and error behavior.

qsc::sphincsplus::sign

byte_vector sign(const byte_vector& message, const secure_vector& private_key);

Purpose: Invoke the native QSC-backed sign operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::sphincsplus::verify

byte_vector verify(const byte_vector& signed_message, const byte_vector& public_key);

Purpose: Invoke the native QSC-backed verify operation. The buffer, parameter and success contracts are defined by its exact signature and implementation.

qsc::sphincsplus::try_generate_keypair

bool try_generate_keypair(keypair& output) noexcept;

Purpose: Non-throwing generate keypair interface. Check the result before reading any output.

qsc::sphincsplus::try_sign

bool try_sign(const byte_vector& message, const secure_vector& private_key, byte_vector& signed_message) noexcept;

Purpose: Non-throwing sign interface. Check the result before reading any output.

qsc::sphincsplus::try_verify

bool try_verify(const byte_vector& signed_message, const byte_vector& public_key, byte_vector& message) noexcept;

Purpose: Non-throwing verify interface. Check the result before reading any output.

22.33 qsccpp_tls.h — 118 callable declarations

TLS 1.3 configuration, connection, listener and server methods.

qsc::tls::status_string

const char* status_string(qsc_tls_socket_status status) noexcept;

Purpose: Converts a QSC TLS socket status code to text

Parameters: status: The QSC TLS socket status code.

Return: Returns a constant status string. /

qsc::tls::default_socket_options

qsc_tls_socket_options default_socket_options() noexcept;

Purpose: Creates a default QSC TLS socket option structure

Return: Returns the initialized socket options. /

qsc::tls::default_ticket_policy

qsc_tls_socket_ticket_policy default_ticket_policy() noexcept;

Purpose: Creates a default QSC TLS session-ticket policy structure

Return: Returns the initialized ticket policy. /

qsc::tls::context::context

context() noexcept;

Purpose: Constructs a context instance; inspect subsequent readiness/status calls before use.

qsc::tls::context::~context

~context();

Purpose: Destructor for context; releases native resources according to the owner lifecycle.

qsc::tls::context::native

const qsc_tls_socket_context* native() const noexcept;

Purpose: Public native operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::native

qsc_tls_socket_context* native() noexcept;

Purpose: Public native operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::is_initialized

bool is_initialized() const noexcept;

Purpose: Public is initialized operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::clear

void clear() noexcept;

Purpose: Public clear operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::try_set_default_client_policy

bool try_set_default_client_policy() noexcept;

Purpose: Non-throwing set default client policy interface. Check the result before reading any output.

qsc::tls::context::try_set_default_server_policy

bool try_set_default_server_policy() noexcept;

Purpose: Non-throwing set default server policy interface. Check the result before reading any output.

qsc::tls::context::try_set_strict_policy

bool try_set_strict_policy() noexcept;

Purpose: Non-throwing set strict policy interface. Check the result before reading any output.

qsc::tls::context::try_set_development_policy

bool try_set_development_policy() noexcept;

Purpose: Non-throwing set development policy interface. Check the result before reading any output.

qsc::tls::context::try_set_mlkem_hybrid_policy

bool try_set_mlkem_hybrid_policy() noexcept;

Purpose: Non-throwing set mlkem hybrid policy interface. Check the result before reading any output.

qsc::tls::context::try_set_experimental_pqc_policy

bool try_set_experimental_pqc_policy() noexcept;

Purpose: Non-throwing set experimental pqc policy interface. Check the result before reading any output.

qsc::tls::context::set_default_client_policy

void set_default_client_policy();

Purpose: Apply the default client policy configuration; validate native state and subsequent call results.

qsc::tls::context::set_default_server_policy

void set_default_server_policy();

Purpose: Apply the default server policy configuration; validate native state and subsequent call results.

qsc::tls::context::set_strict_policy

void set_strict_policy();

Purpose: Apply the strict policy configuration; validate native state and subsequent call results.

qsc::tls::context::set_development_policy

void set_development_policy();

Purpose: Apply the development policy configuration; validate native state and subsequent call results.

qsc::tls::context::set_mlkem_hybrid_policy

void set_mlkem_hybrid_policy();

Purpose: Apply the mlkem hybrid policy configuration; validate native state and subsequent call results.

qsc::tls::context::set_experimental_pqc_policy

void set_experimental_pqc_policy();

Purpose: Apply the experimental pqc policy configuration; validate native state and subsequent call results.

qsc::tls::context::try_set_socket_options

bool try_set_socket_options(const qsc_tls_socket_options& options) noexcept;

Purpose: Non-throwing set socket options interface. Check the result before reading any output.

qsc::tls::context::set_socket_options

void set_socket_options(const qsc_tls_socket_options& options);

Purpose: Apply the socket options configuration; validate native state and subsequent call results.

qsc::tls::context::try_set_cipher_suites

bool try_set_cipher_suites(const std::vector<qsc_tls_cipher_suite>& suites) noexcept;

Purpose: Configure preferred TLS cipher suites in negotiation order

Parameters: suites: Required ordered suite list (caller-owned).

Return: True if the native TLS policy accepts the list; false otherwise

qsc::tls::context::set_cipher_suites

void set_cipher_suites(const std::vector<qsc_tls_cipher_suite>& suites);

Purpose: Set cipher suite preference or throw on rejection

Parameters: suites: Ordered suite list; must be nonempty and within QSC limits. \throws crypto_exception for native policy rejection.

qsc::tls::context::try_set_named_groups

bool try_set_named_groups(const std::vector<qsc_tls_named_group>& groups) noexcept;

Purpose: Configure allowed TLS named groups in preference order

Parameters: groups: Caller-owned ordered groups.

Return: True if accepted by native policy

qsc::tls::context::set_named_groups

void set_named_groups(const std::vector<qsc_tls_named_group>& groups);

Purpose: Configure named groups or throw on native policy failure

Parameters: groups: Ordered group list. \throws crypto_exception when the QSC policy rejects the list.

qsc::tls::context::try_set_signature_schemes

bool try_set_signature_schemes(const std::vector<qsc_tls_signature_scheme>& schemes) noexcept;

Purpose: Configure allowed TLS signature schemes

Parameters: schemes: Caller-owned ordered signature schemes.

Return: True if accepted by native policy

qsc::tls::context::set_signature_schemes

void set_signature_schemes(const std::vector<qsc_tls_signature_scheme>& schemes);

Purpose: Configure signature schemes or throw

Parameters: schemes: Ordered signature scheme list. \throws crypto_exception for QSC policy rejection.

qsc::tls::context::try_set_ticket_policy

bool try_set_ticket_policy(const qsc_tls_socket_ticket_policy& policy) noexcept;

Purpose: Non-throwing set ticket policy interface. Check the result before reading any output.

qsc::tls::context::set_ticket_policy

void set_ticket_policy(const qsc_tls_socket_ticket_policy& policy);

Purpose: Apply the ticket policy configuration; validate native state and subsequent call results.

qsc::tls::context::try_set_alpn_protocols

bool try_set_alpn_protocols(const std::vector<std::string>& protocols, bool required) noexcept;

Purpose: Non-throwing set alpn protocols interface. Check the result before reading any output.

qsc::tls::context::set_alpn_protocols

void set_alpn_protocols(const std::vector<std::string>& protocols, bool required);

Purpose: Apply the alpn protocols configuration; validate native state and subsequent call results.

qsc::tls::context::try_clear_alpn_protocols

bool try_clear_alpn_protocols() noexcept;

Purpose: Non-throwing clear alpn protocols interface. Check the result before reading any output.

qsc::tls::context::clear_alpn_protocols

void clear_alpn_protocols();

Purpose: Public clear alpn protocols operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::try_set_sni_required

bool try_set_sni_required(bool required) noexcept;

Purpose: Non-throwing set sni required interface. Check the result before reading any output.

qsc::tls::context::set_sni_required

void set_sni_required(bool required);

Purpose: Apply the sni required configuration; validate native state and subsequent call results.

qsc::tls::context::try_set_client_auth

bool try_set_client_auth(bool request_client_auth, bool require_client_auth) noexcept;

Purpose: Non-throwing set client auth interface. Check the result before reading any output.

qsc::tls::context::set_client_auth

void set_client_auth(bool request_client_auth, bool require_client_auth);

Purpose: Apply the client auth configuration; validate native state and subsequent call results.

qsc::tls::context::try_load_trust_anchor_file

bool try_load_trust_anchor_file(const std::string& path, bool self_signed) noexcept;

Purpose: Non-throwing load trust anchor file interface. Check the result before reading any output.

qsc::tls::context::load_trust_anchor_file

void load_trust_anchor_file(const std::string& path, bool self_signed);

Purpose: Public load trust anchor file operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::try_load_trust_anchor_bundle_file

bool try_load_trust_anchor_bundle_file(const std::string& path, bool self_signed) noexcept;

Purpose: Non-throwing load trust anchor bundle file interface. Check the result before reading any output.

qsc::tls::context::load_trust_anchor_bundle_file

void load_trust_anchor_bundle_file(const std::string& path, bool self_signed);

Purpose: Public load trust anchor bundle file operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::try_load_client_identity_files

bool try_load_client_identity_files(const std::string& certificate_chain_path, const std::string& private_key_path, qsc_tls_signature_scheme verify_scheme) noexcept;

Purpose: Load client certificate chain and private key for mutual TLS

Parameters: certificate_chain_path: Certificate chain file path.; private_key_path: Key file path; never copied into C++ strings beyond parameter lifetime.; verify_scheme: CertificateVerify signature scheme.

Return: True on native success; client identity is owned by TLS context

qsc::tls::context::load_client_identity_files

void load_client_identity_files(const std::string& certificate_chain_path, const std::string& private_key_path, qsc_tls_signature_scheme verify_scheme);

Purpose: Load mTLS client identity or throw

Parameters: certificate_chain_path: Certificate chain file.; private_key_path: Secret private key file.; verify_scheme: Signature scheme. \throws crypto_exception on load or policy failure.

qsc::tls::context::try_set_session_ticket

bool try_set_session_ticket(const qsc_tls_session_ticket& ticket) noexcept;

Purpose: Supply a validated resumption ticket to this TLS context

Parameters: ticket: Caller-owned ticket; QSC copies its retained state.

Return: True if ticket validity and policy checks succeed

qsc::tls::context::clear_session_ticket

void clear_session_ticket() noexcept;

Purpose: Remove and erase the current resumption ticket

qsc::tls::context::try_set_log_callback

bool try_set_log_callback(qsc_tls_socket_log_callback callback, void* state) noexcept;

Purpose: Set a structured TLS logging callback; callback state remains caller-owned

Parameters: callback: QSC TLS logger function; may be null to disable.; state: Callback state valid until callback removed/disposal.

Return: True on native success

qsc::tls::context::try_set_client_authorization

bool try_set_client_authorization(qsc_tls_client_authorization_callback callback, void* state, bool required) noexcept;

Purpose: Enforce post-certificate application client authorization

Parameters: callback: Application authorization callback; caller-owned lifetime.; state: Callback context valid through connection operations.; required: Reject clients unless the callback authorizes them.

Return: True if accepted by native context

qsc::tls::context::try_set_psk_lookup_callback

bool try_set_psk_lookup_callback(qsc_tls_psk_lookup_callback callback, void* state) noexcept;

Purpose: Configure server-side ticket PSK lookup

Parameters: callback: Lookup callback or null.; state: Backend state valid through callbacks.

Return: True if accepted; thread safety follows backend callback

qsc::tls::context::try_load_crl_file

bool try_load_crl_file(const std::string& path) noexcept;

Purpose: Non-throwing load crl file interface. Check the result before reading any output.

qsc::tls::context::load_crl_file

void load_crl_file(const std::string& path);

Purpose: Public load crl file operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::try_load_server_identity_files

bool try_load_server_identity_files(const std::string& certificate_chain_path, const std::string& private_key_path, qsc_tls_signature_scheme verify_scheme) noexcept;

Purpose: Non-throwing load server identity files interface. Check the result before reading any output.

qsc::tls::context::load_server_identity_files

void load_server_identity_files(const std::string& certificate_chain_path, const std::string& private_key_path, qsc_tls_signature_scheme verify_scheme);

Purpose: Public load server identity files operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::context::try_add_server_identity_files

bool try_add_server_identity_files(const std::string& hostname, const std::string& certificate_chain_path, const std::string& private_key_path, qsc_tls_signature_scheme verify_scheme) noexcept;

Purpose: Non-throwing add server identity files interface. Check the result before reading any output.

qsc::tls::context::add_server_identity_files

void add_server_identity_files(const std::string& hostname, const std::string& certificate_chain_path, const std::string& private_key_path, qsc_tls_signature_scheme verify_scheme);

Purpose: Public add server identity files operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::connection

connection() noexcept;

Purpose: Constructs a connection instance; inspect subsequent readiness/status calls before use.

qsc::tls::connection::~connection

~connection();

Purpose: Destructor for connection; releases native resources according to the owner lifecycle.

qsc::tls::connection::native

const qsc_tls_socket_connection* native() const noexcept;

Purpose: Public native operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::native

qsc_tls_socket_connection* native() noexcept;

Purpose: Public native operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::clear

void clear() noexcept;

Purpose: Public clear operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::try_connect_host

bool try_connect_host(const context& ctx, const std::string& hostname, const std::string& service) noexcept;

Purpose: Non-throwing connect host interface. Check the result before reading any output.

qsc::tls::connection::connect_host

void connect_host(const context& ctx, const std::string& hostname, const std::string& service);

Purpose: Public connect host operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::try_connect_host_ex

bool try_connect_host_ex(const context& ctx, const std::string& hostname, const std::string& service, const qsc_tls_session_ticket* ticket, bool enable_early_data) noexcept;

Purpose: Connect with a supplied resumption ticket and optional 0-RTT request

Parameters: ctx: Owning TLS policy context; must outlive connection handshake.; hostname: Hostname for peer identification.; service: Service/port string.; ticket: Optional QSC ticket, or nullptr for a fresh handshake.; enable_early_data: Request 0-RTT if the QSC profile allows it.

Return: True if QSC completed the requested client connection

qsc::tls::connection::try_key_update

bool try_key_update(bool request_peer_update) noexcept;

Purpose: Request TLS 1.3 traffic-key update on an established connection

Parameters: request_peer_update: Request a reciprocal key update when true.

Return: True if the native TLS connection accepts the update

qsc::tls::connection::key_update

void key_update(bool request_peer_update);

Purpose: Request TLS key update or throw on error

Parameters: request_peer_update: Request peer update. \throws crypto_exception on failure or invalid state.

qsc::tls::connection::try_get_session_ticket

bool try_get_session_ticket(qsc_tls_session_ticket& ticket) const noexcept;

Purpose: Retrieve the latest resumption ticket

Parameters: ticket: Caller-owned output; caller must dispose sensitive native ticket state.

Return: True if a valid ticket is available

qsc::tls::connection::clear_session_ticket

void clear_session_ticket() noexcept;

Purpose: Erase a previously stored native session ticket in this connection

qsc::tls::connection::try_set_socket_options

bool try_set_socket_options(const qsc_tls_socket_options& options) noexcept;

Purpose: Configure live connection socket options

Parameters: options: Native socket timeout/network options.

Return: True if accepted by this connection

qsc::tls::connection::try_set_log_callback

bool try_set_log_callback(qsc_tls_socket_log_callback callback, void* state) noexcept;

Purpose: Configure live connection logging callback

Parameters: callback: QSC TLS logger; caller-owned lifetime.; state: Callback state valid through callback use.

Return: True on native success

qsc::tls::connection::try_send

bool try_send(const byte_vector& input, std::size_t& written) noexcept;

Purpose: Non-throwing send interface. Check the result before reading any output.

qsc::tls::connection::send

std::size_t send(const byte_vector& input);

Purpose: Public send operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::try_receive

bool try_receive(byte_vector& output, std::size_t& read) noexcept;

Purpose: Non-throwing receive interface. Check the result before reading any output.

qsc::tls::connection::receive

byte_vector receive(std::size_t capacity);

Purpose: Public receive operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::try_send_frame

bool try_send_frame(const byte_vector& input) noexcept;

Purpose: Non-throwing send frame interface. Check the result before reading any output.

qsc::tls::connection::send_frame

void send_frame(const byte_vector& input);

Purpose: Public send frame operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::try_receive_frame

bool try_receive_frame(byte_vector& output, std::size_t& read) noexcept;

Purpose: Non-throwing receive frame interface. Check the result before reading any output.

qsc::tls::connection::receive_frame

byte_vector receive_frame(std::size_t capacity);

Purpose: Public receive frame operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::try_shutdown

bool try_shutdown() noexcept;

Purpose: Non-throwing shutdown interface. Check the result before reading any output.

qsc::tls::connection::shutdown

void shutdown();

Purpose: Public shutdown operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::try_cancel

bool try_cancel() noexcept;

Purpose: Non-throwing cancel interface. Check the result before reading any output.

qsc::tls::connection::cancel

void cancel();

Purpose: Public cancel operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::is_connected

bool is_connected() const noexcept;

Purpose: Public is connected operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::is_handshake_complete

bool is_handshake_complete() const noexcept;

Purpose: Public is handshake complete operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::negotiated_cipher_suite

qsc_tls_cipher_suite negotiated_cipher_suite() const noexcept;

Purpose: Public negotiated cipher suite operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::negotiated_group

qsc_tls_named_group negotiated_group() const noexcept;

Purpose: Public negotiated group operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::negotiated_signature_scheme

qsc_tls_signature_scheme negotiated_signature_scheme() const noexcept;

Purpose: Public negotiated signature scheme operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::connection::try_peer_info

bool try_peer_info(peer_info& info) const noexcept;

Purpose: Non-throwing peer info interface. Check the result before reading any output.

qsc::tls::connection::get_peer_info

peer_info get_peer_info() const;

Purpose: Query get peer info metadata using the selected algorithm, profile or object; validate returned size or status.

qsc::tls::connection::try_selected_alpn

bool try_selected_alpn(std::string& protocol) const noexcept;

Purpose: Non-throwing selected alpn interface. Check the result before reading any output.

qsc::tls::connection::selected_alpn

std::string selected_alpn() const;

Purpose: Public selected alpn operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::listener::listener

listener() noexcept;

Purpose: Constructs a listener instance; inspect subsequent readiness/status calls before use.

qsc::tls::listener::~listener

~listener();

Purpose: Destructor for listener; releases native resources according to the owner lifecycle.

qsc::tls::listener::try_bind

bool try_bind(const context& ctx, const std::string& address, std::uint16_t port, qsc_socket_address_families family) noexcept;

Purpose: Non-throwing bind interface. Check the result before reading any output.

qsc::tls::listener::bind

void bind(const context& ctx, const std::string& address, std::uint16_t port, qsc_socket_address_families family);

Purpose: Public bind operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::listener::try_set_socket_options

bool try_set_socket_options(const qsc_tls_socket_options& options) noexcept;

Purpose: Configure listener socket options before binding

Parameters: options: QSC listener options.

Return: True if QSC accepts the options

qsc::tls::listener::try_accept

bool try_accept(connection& conn) noexcept;

Purpose: Non-throwing accept interface. Check the result before reading any output.

qsc::tls::listener::accept

void accept(connection& conn);

Purpose: Public accept operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::listener::close

void close() noexcept;

Purpose: Public close operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::listener::native

const qsc_tls_socket_listener* native() const noexcept;

Purpose: Public native operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::listener::native

qsc_tls_socket_listener* native() noexcept;

Purpose: Public native operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::server::server

server() noexcept;

Purpose: Constructs a server instance; inspect subsequent readiness/status calls before use.

qsc::tls::server::~server

~server();

Purpose: Destructor for server; releases native resources according to the owner lifecycle.

qsc::tls::server::server

server(server&& other) noexcept;

Purpose: Constructs a server instance; inspect subsequent readiness/status calls before use.

qsc::tls::server::operator=

server& operator=(server&& other) noexcept;

Purpose: C++ operator defined by the declaration. Its semantics and ownership are governed by the operand types.

qsc::tls::server::try_configure

bool try_configure(const context& ctx, const std::string& address, std::uint16_t port, qsc_socket_address_families family) noexcept;

Purpose: Non-throwing configure interface. Check the result before reading any output.

qsc::tls::server::configure

void configure(const context& ctx, const std::string& address, std::uint16_t port, qsc_socket_address_families family);

Purpose: Public configure operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::server::try_set_callbacks

bool try_set_callbacks(qsc_tls_socket_server_connect_callback onconnect, qsc_tls_socket_server_receive_callback onreceive, qsc_tls_socket_server_disconnect_callback ondisconnect, qsc_tls_socket_server_error_callback onerror, void* state) noexcept;

Purpose: Register server callbacks for the active server lifetime

Parameters: onconnect: Callback for accepted connections.; onreceive: Callback for received bytes.; ondisconnect: Callback for connection removal.; onerror: Callback for connection errors.; state: Caller-owned callback state, kept alive through stop/clear.

Return: True if QSC accepts callbacks; caller must serialize changes with active server work

qsc::tls::server::try_set_max_clients

bool try_set_max_clients(std::size_t max_clients) noexcept;

Purpose: Non-throwing set max clients interface. Check the result before reading any output.

qsc::tls::server::set_max_clients

void set_max_clients(std::size_t max_clients);

Purpose: Apply the max clients configuration; validate native state and subsequent call results.

qsc::tls::server::try_start

bool try_start() noexcept;

Purpose: Non-throwing start interface. Check the result before reading any output.

qsc::tls::server::start

void start();

Purpose: Public start operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::server::try_start_concurrent

bool try_start_concurrent() noexcept;

Purpose: Non-throwing start concurrent interface. Check the result before reading any output.

qsc::tls::server::start_concurrent

void start_concurrent();

Purpose: Public start concurrent operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::server::stop

void stop() noexcept;

Purpose: Public stop operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::server::clear

void clear() noexcept;

Purpose: Public clear operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::server::native

const qsc_tls_socket_server* native() const noexcept;

Purpose: Public native operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::tls::server::native

qsc_tls_socket_server* native() noexcept;

Purpose: Public native operation; review the declared signature and its QSC-backed implementation for preconditions.

22.34 qsccpp_utils.h — 9 callable declarations

Public utils C++ functions and object methods.

qsc::utils::data_or_null

const byte* data_or_null(const byte_vector& data) noexcept;

Purpose: Public data or null operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::utils::data_or_null

byte* data_or_null(byte_vector& data) noexcept;

Purpose: Public data or null operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::utils::data_or_null

const char* data_or_null(const std::string& data) noexcept;

Purpose: Public data or null operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::utils::data_or_null

char* data_or_null(std::string& data) noexcept;

Purpose: Public data or null operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::utils::is_valid_read_buffer

bool is_valid_read_buffer(const byte* data, std::size_t length) noexcept;

Purpose: Public is valid read buffer operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::utils::is_valid_write_buffer

bool is_valid_write_buffer(byte* data, std::size_t length) noexcept;

Purpose: Public is valid write buffer operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::utils::is_equal

bool is_equal(const byte_vector& left, const byte_vector& right) noexcept;

Purpose: Public is equal operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::utils::secure_clear

void secure_clear(byte_vector& data) noexcept;

Purpose: Public secure clear operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::utils::resize

bool resize(byte_vector& data, std::size_t length) noexcept;

Purpose: Public resize operation; review the declared signature and its QSC-backed implementation for preconditions.

22.35 qsccpp_x509.h — 39 callable declarations

Certificate, trust-store and identity inspection RAII objects.

qsc::x509::status_string

const char* status_string(qsc_x509w_status status) noexcept;

Purpose: Converts a QSC X.509 wrapper status to text

Parameters: status: The QSC X.509 wrapper status code.

Return: Returns a constant status string. /

qsc::x509::verify_status_string

const char* verify_status_string(qsc_x509_verify_status status) noexcept;

Purpose: Converts a QSC X.509 verification status to text

Parameters: status: The QSC X.509 verification status code.

Return: Returns a constant verification-status string. /

qsc::x509::certificate::certificate

certificate() noexcept;

Purpose: Allocate stable native certificate storage; valid even if allocation fails

Details: Inspect is_loaded() after decoding; no exceptions are thrown

qsc::x509::certificate::~certificate

~certificate();

Purpose: Release native certificate state and DER backing memory

qsc::x509::certificate::certificate

certificate(certificate&& other) noexcept;

Purpose: Transfer native certificate and owned DER backing storage

Parameters: other: Previous owner; left unloaded and without native storage.

qsc::x509::certificate::operator=

certificate& operator=(certificate&& other) noexcept;

Purpose: Release current state and transfer ownership from another certificate

Parameters: other: Previous owner; may be this object.

Return: This object, whose native addresses remain stable

qsc::x509::certificate::is_loaded

bool is_loaded() const noexcept;

Purpose: Determine whether a valid decoded certificate is currently loaded

Return: True after successful DER or PEM decode, false otherwise

qsc::x509::certificate::native

const qsc_x509_certificate* native() const noexcept;

Purpose: Borrow the decoded native certificate without transferring ownership

Return: Pointer valid until clear, destruction or move; null after move

qsc::x509::certificate::native

qsc_x509_certificate* native() noexcept;

Purpose: Borrow mutable native state without transferring ownership

Return: Pointer with the same lifetime rules as the const overload

qsc::x509::certificate::clear

void clear() noexcept;

Purpose: Clear native state and release any owned DER backing storage

Details: Safe on unloaded or moved-from certificates; not thread-safe

qsc::x509::certificate::try_load_pem

bool try_load_pem(const std::string& pem) noexcept;

Purpose: Decode PEM into owned certificate storage

Parameters: pem: Input PEM text, borrowed for this call.

Return: True on successful decoding; false on invalid input/allocation

qsc::x509::certificate::try_load_der

bool try_load_der(const byte_vector& der) noexcept;

Purpose: Decode DER after privately copying the input to stable backing storage

Parameters: der: Certificate DER, borrowed for this call; may expire on return.

Return: True on success, false on parse/allocation failure

qsc::x509::certificate::load_pem

void load_pem(const std::string& pem);

Purpose: Load PEM or raise crypto_exception on invalid input/decoding error

Parameters: pem: Input PEM, borrowed during the call only.

qsc::x509::certificate::load_der

void load_der(const byte_vector& der);

Purpose: Load DER or raise crypto_exception on invalid input/decoding error

Parameters: der: Caller-owned DER, privately copied into this object.

qsc::x509::certificate::subject_string

std::string subject_string() const;

Purpose: Return a formatted subject distinguished name

Return: Subject string; throws crypto_exception when unavailable

qsc::x509::certificate::issuer_string

std::string issuer_string() const;

Purpose: Return a formatted issuer distinguished name

Return: Issuer string; throws crypto_exception when unavailable

qsc::x509::certificate::subject_common_name

std::string subject_common_name() const;

Purpose: Read the subject common name

Return: Subject common name; throws when absent/unloaded

qsc::x509::certificate::dns_name_count

std::size_t dns_name_count() const noexcept;

Purpose: Report the number of subject DNS names

Return: Zero for unloaded certificates

qsc::x509::certificate::dns_name

std::string dns_name(std::size_t index) const;

Purpose: Read one DNS SAN from a loaded certificate

Parameters: index: Zero-based DNS SAN index.

Return: DNS name; throws crypto_exception on missing index

qsc::x509::certificate::match_hostname

bool match_hostname(const std::string& hostname) const noexcept;

Purpose: Match a hostname to a decoded certificate's name constraints

Parameters: hostname: Candidate hostname; must not be empty.

Return: True only when native hostname check succeeds

qsc::x509::certificate::export_der

byte_vector export_der() const;

Purpose: Export certificate DER into a caller-owning C++ buffer

Return: DER bytes; throws on invalid/unloaded certificate

qsc::x509::certificate::export_pem

std::string export_pem() const;

Purpose: Export certificate PEM into a C++ string

Return: PEM text; throws on native encoding failure

qsc::x509::make_profile

qsc_x509w_profile make_profile(qsc_x509w_profile_preset preset) noexcept;

Purpose: Construct a QSC X.509 policy with an explicit validation preset

Parameters: preset: Trusted configuration preset (e.g. TLS_SERVER).

Return: Initialized policy to supply to certificate verification

qsc::x509::trust_store::trust_store

trust_store() noexcept;

Purpose: Allocate and initialize the native QSC trust store

Details: Allocation failures result in an invalid store

qsc::x509::trust_store::~trust_store

~trust_store();

Purpose: Release certificates, CRLs and native store allocation

qsc::x509::trust_store::trust_store

trust_store(trust_store&& other) noexcept;

Purpose: Transfer the native allocation with stable internal pointers

Parameters: other: Previous owner, left invalid.

qsc::x509::trust_store::operator=

trust_store& operator=(trust_store&& other) noexcept;

Purpose: Replace this store by moving another one

Parameters: other: Previous owner, left invalid.

Return: This store

qsc::x509::trust_store::valid

bool valid() const noexcept;

Purpose: Check whether the native allocation is initialized

Return: True when the native state exists

qsc::x509::trust_store::native

const qsc_x509w_trust_store* native() const noexcept;

Purpose: Obtain a borrowed read-only native store pointer

Return: Pointer valid until this store is cleared/destroyed

qsc::x509::trust_store::anchor_count

std::size_t anchor_count() const noexcept;

Purpose: Return current trusted certificate count

Return: Number of configured trust anchors

qsc::x509::trust_store::clear

void clear() noexcept;

Purpose: Remove current anchors and CRLs without freeing the store

qsc::x509::trust_store::add_anchor

qsc_x509w_status add_anchor(const certificate& cert, bool self_signed) noexcept;

Purpose: Clone a decoded certificate into explicitly configured trust anchors

Parameters: cert: Loaded certificate borrowed for this call only.; self_signed: Declares a self-signed trust anchor when true.

Return: Native status; does not imply that this anchor is trustworthy

qsc::x509::trust_store::add_anchor_bundle

qsc_x509w_status add_anchor_bundle(const byte_vector& input, bool self_signed) noexcept;

Purpose: Add trusted PEM/DER certificate bundle from caller memory

Parameters: input: Encoded certificates; nonempty.; self_signed: Treat included certificates as self-signed anchors.

Return: Typed native status

qsc::x509::trust_store::add_crl

qsc_x509w_status add_crl(const byte_vector& input) noexcept;

Purpose: Load a revocation list from caller-owned DER/PEM bytes

Parameters: input: Encoded CRL, must be nonempty.

Return: Typed native status

qsc::x509::trust_store::verify_peer

qsc_x509w_status verify_peer(const certificate& cert, const qsc_x509w_profile& profile, qsc_x509w_result& result) const noexcept;

Purpose: Verify a single peer certificate against this store and policy

Parameters: cert: Loaded peer certificate.; profile: Caller-selected verification policy.; result: Detailed native status and diagnostic output.

Return: Typed native validation status; reject any nonsuccess

qsc::x509::load_certificate_pem

certificate load_certificate_pem(const std::string& pem);

Purpose: Public load certificate pem operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::x509::load_certificate_der

certificate load_certificate_der(const byte_vector& der);

Purpose: Public load certificate der operation; review the declared signature and its QSC-backed implementation for preconditions.

qsc::x509::try_load_certificate_pem

bool try_load_certificate_pem(const std::string& pem, certificate& cert) noexcept;

Purpose: Non-throwing load certificate pem interface. Check the result before reading any output.

qsc::x509::try_load_certificate_der

bool try_load_certificate_der(const byte_vector& der, certificate& cert) noexcept;

Purpose: Non-throwing load certificate der interface. Check the result before reading any output.

22.36 qsccpp_x509_issuance.h — 38 callable declarations

CSR, issuance, signed CRL and protected PKCS#12 interfaces.

qsc::x509::certificate_request::certificate_request

certificate_request() noexcept;

Purpose: Allocate and initialize a native QSC CSR

qsc::x509::certificate_request::~certificate_request

~certificate_request();

Purpose: Destroy the native CSR and release its backing buffers

qsc::x509::certificate_request::certificate_request

certificate_request(certificate_request&& other) noexcept;

Purpose: Transfer stable native ownership

Parameters: other: Previous owner, left invalid.

qsc::x509::certificate_request::operator=

certificate_request& operator=(certificate_request&& other) noexcept;

Purpose: Replace with another CSR and transfer its ownership

Parameters: other: Previous owner.

Return: This object

qsc::x509::certificate_request::valid

bool valid() const noexcept;

Purpose: Return true if native CSR storage is allocated

qsc::x509::certificate_request::native

const qsc_x509_csr* native() const noexcept;

Purpose: Borrow immutable native CSR (may be nullptr)

qsc::x509::certificate_request::clear

void clear() noexcept;

Purpose: Wipe/release native CSR allocations, retaining object storage

qsc::x509::certificate_request::create

qsc_x509w_status create(const qsc_x509_name& subject, const qsc_x509_subject_public_key_info& spki, const qsc_x509_algorithm_identifier& algorithm) noexcept;

Purpose: Construct CSR fields from QSC native name, SPKI and signing algorithm

Parameters: subject: Borrowed subject name.; spki: Borrowed subject public key.; algorithm: Signature algorithm identifier.

Return: Native result; no signing or verification is performed

qsc::x509::certificate_request::add_dns_name

qsc_x509w_status add_dns_name(const std::string& hostname) noexcept;

Purpose: Attach a DNS SAN to this request (before signing)

Parameters: hostname: NUL-free caller-owned DNS hostname.

Return: Native status, or invalid-input for unallocated CSR

qsc::x509::certificate_request::try_load_der

bool try_load_der(const byte_vector& der) noexcept;

Purpose: Decode CSR DER; no trust decision implied

Parameters: der: Nonempty encoded request bytes.

Return: True for a valid DER structure

qsc::x509::certificate_request::try_load_pem

bool try_load_pem(const std::string& pem) noexcept;

Purpose: Decode CSR PEM; no trust decision implied

Parameters: pem: Nonempty PEM text.

Return: True on successful decoding

qsc::x509::certificate_request::verify

bool verify() const noexcept;

Purpose: Check a parsed CSR's self-signature using the native verifier

Return: True for successful native signature validation

qsc::x509::certificate_request::sign_der

qsc_x509w_status sign_der(qsc_x509_certificate_sign_callback signcallback, void* context, byte_vector& der) const noexcept;

Purpose: Sign CSR via borrowed QSC callback and export a DER request

Parameters: signcallback: Native C callback, must not throw.; context: Callback state, borrowed for the duration.; der: Caller-owned output, erased on failure.

Return: Native status; rejects unallocated CSR and null callback

qsc::x509::issue_from_csr

qsc_x509w_status issue_from_csr(const certificate_request& request, const certificate& issuer, const qsc_x509_algorithm_identifier& signature_algorithm, const byte_vector& serial, const qsc_x509_validity& validity, std::uint32_t profile, std::uint32_t policy_flags, qsc_x509_certificate_sign_callback signcallback, void* context, byte_vector& output) noexcept;

Purpose: Bounded certificate issuance from a validated CSR and explicit issuer

Details: Does not silently select a profile or trust anchor. Caller must enforce issuer privileges, serial uniqueness, validity bounds and signing-key custody

Parameters: request: Borrowed initialized CSR.; issuer: Loaded issuer certificate, never an implicit trust anchor.; signature_algorithm: Native algorithm identifier.; serial: Nonempty positive serial bytes.; validity: Native date interval; caller must validate policy.; profile: Issuance profile flags; policy_flags: Requested extension policy.; signcallback: Nonthrowing native signing callback; context: Borrowed callback state.; output: DER certificate, cleared on failure.

Return: Exact native result, including profile/policy errors

qsc::x509::crl_builder::crl_builder

crl_builder() noexcept;

Purpose: Allocate and initialize stable native builder storage

qsc::x509::crl_builder::~crl_builder

~crl_builder();

Purpose: Clear and release native builder storage

qsc::x509::crl_builder::crl_builder

crl_builder(crl_builder&& other) noexcept;

Purpose: Transfer builder ownership; source becomes invalid

Parameters: other: Source object.

qsc::x509::crl_builder::operator=

crl_builder& operator=(crl_builder&& other) noexcept;

Purpose: Replace native ownership, cleaning previous builder

Parameters: other: Source object, left invalid.

Return: This object

qsc::x509::crl_builder::valid

bool valid() const noexcept;

Purpose: Test native allocation success

Return: True when builder storage exists

qsc::x509::crl_builder::revoked_count

std::size_t revoked_count() const noexcept;

Purpose: Number of added revoked serial numbers

Return: Zero on invalid or empty builder

qsc::x509::crl_builder::clear

void clear() noexcept;

Purpose: Wipe and reinitialize builder while retaining allocation

qsc::x509::crl_builder::configure

qsc_asn1_status configure(const certificate& issuer, const qsc_asn1_time& thisupdate, const qsc_asn1_time& nextupdate, const qsc_x509_algorithm_identifier& algorithm) noexcept;

Purpose: Configure issuer, update times and signature algorithm

Details: Rejects issuer certificates lacking CA or cRLSign authority

Parameters: issuer: Loaded CA certificate, borrowed for this call.; thisupdate: CRL activation time.; nextupdate: Expiration time, not earlier than activation.; algorithm: Exact signature algorithm identifier.

Return: Native ASN.1 status; clears incomplete configuration on failure

qsc::x509::crl_builder::set_crl_number

qsc_asn1_status set_crl_number(const byte_vector& number, bool critical = false) noexcept;

Purpose: Assign the positive, monotonically increasing CRL number

Parameters: number: Nonempty unsigned, big-endian CRL number.; critical: Whether to mark the extension critical.

Return: Exact native ASN.1 status; rejects zero and empty input

qsc::x509::crl_builder::add_revoked

qsc_asn1_status add_revoked(const byte_vector& serial, const qsc_asn1_time& revoked_at) noexcept;

Purpose: Add a revoked certificate serial and effective date

Parameters: serial: Nonzero big-endian serial, copied into builder.; revoked_at: Effective revocation timestamp.

Return: Native result, including duplicate and capacity errors

qsc::x509::crl_builder::sign_der

qsc_asn1_status sign_der(const certificate& issuer, qsc_x509_certificate_sign_callback signcallback, void* context, byte_vector& output) const noexcept;

Purpose: Generate an issuer-validated, signed CRL as DER

Parameters: issuer: Loaded CA certificate used for policy validation.; signcallback: Nonthrowing C signer, borrowed for the call.; context: Borrowed signer context, retained by neither layer.; output: Caller-owned DER bytes; cleared on failure.

Return: Native ASN.1 result; does not imply trust-chain validation

qsc::x509::crl_builder::encode_pem

static qsc_asn1_status encode_pem(const byte_vector& der, std::string& output) noexcept;

Purpose: Encode an already signed DER CRL in PEM armor

Parameters: der: Caller-owned nonempty DER-encoded CRL.; output: Caller-owned PEM text; cleared on failure.

Return: Native ASN.1 status, not signature-verification status

qsc::x509::crl_builder::verify_der

static qsc_x509_crl_verify_status verify_der(const byte_vector& der, const certificate& issuer, const qsc_asn1_time& now, qsc_x509_crl_signature_verify_callback verifycallback, void* context) noexcept;

Purpose: Decode and cryptographically validate a signed DER CRL

Details: Delegates issuer matching, validity intervals, signing authority, and signature verification to QSC. Does not independently establish trust in the supplied issuer; callers must verify the issuer's trust chain

Parameters: der: Encoded CRL, borrowed for this call only.; issuer: Verified issuer CA certificate, borrowed for this call.; now: Caller-selected validation time.; verifycallback: C signature-verification callback, must not throw.; context: Required nonnull callback state, borrowed and never retained.

Return: Typed QSC CRL verification result, or invalid input/CRL

qsc::x509::pkcs12_bundle::pkcs12_bundle

pkcs12_bundle() noexcept;

Purpose: Allocate/init native bundle (may fail; inspect valid())

qsc::x509::pkcs12_bundle::~pkcs12_bundle

~pkcs12_bundle();

Purpose: Clear sensitive key and certificate allocations

qsc::x509::pkcs12_bundle::pkcs12_bundle

pkcs12_bundle(pkcs12_bundle&& other) noexcept;

Purpose: Transfer stable bundle ownership

Parameters: other: Previous owner, left invalid.

qsc::x509::pkcs12_bundle::operator=

pkcs12_bundle& operator=(pkcs12_bundle&& other) noexcept;

Purpose: Transfer ownership after releasing old bundle

Parameters: other: Previous owner.

Return: This instance

qsc::x509::pkcs12_bundle::valid

bool valid() const noexcept;

Purpose: Indicate successful allocation of native bundle storage

qsc::x509::pkcs12_bundle::is_loaded

bool is_loaded() const noexcept;

Purpose: Report whether the last parse completed and MAC validated

qsc::x509::pkcs12_bundle::certificate_count

std::size_t certificate_count() const noexcept;

Purpose: Number of decoded certificates, or zero if invalid/unloaded

qsc::x509::pkcs12_bundle::native

const qsc_x509_pkcs12_bundle* native() const noexcept;

Purpose: Borrow validated native bundle, valid until clear or destruction

Return: Null when the bundle has not passed parsing and MAC verification

qsc::x509::pkcs12_bundle::clear

void clear() noexcept;

Purpose: Clear parsed keys/certificates while keeping bundle allocated

qsc::x509::pkcs12_bundle::try_parse

bool try_parse(const byte_vector& der, const std::string& password) noexcept;

Purpose: Parse and validate PKCS#12 DER with caller-owned password

Parameters: der: Nonempty DER bytes; password: Borrowed UTF-8 password.

Return: True only if QSC parses/decrypts and verifies MAC/key association

qsc::x509::export_private_key_pkcs8_der

qsc_x509w_status export_private_key_pkcs8_der(const qsc_x509_private_key& key, secure_vector& output) noexcept;

Purpose: Export native private key to PKCS#8 DER in secure memory

Parameters: key: Borrowed native key; output: Secret encoded key bytes, securely cleared on failure.

Return: Native wrapper status; caller owns all exported bytes and must erase promptly

23: Build and Release Acceptance Checklist

A completed implementation of QSCCPP must satisfy the following acceptance criteria for every platform and cryptographic profile. The checklist is an engineering release gate, not evidence that the uploaded archive has passed those tests.

- Pin the matching native QSC revision, compiler/CRT, target architecture, CPU ISA and C++17 toolchain.
- Supply a correct parent build target or equivalent Visual Studio solution and ensure QSCCPP_QSC_TARGET resolves.
- Audit the public 659-declaration C++ header surface used by each application and its feature guards.
- Qualify status_to_string, crypto_exception, try_ result reporting and output failure behavior.
- Verify secure_vector zeroization on reallocation, clear, move and exception paths; do not misuse allocator equality.
- Verify SHA2/SHA3 streams after reset/finalize and no concurrent use of mutable state.
- Run known-answer, tampering, invalid-size and authentication-failure tests for each selected cipher and MAC.
- Bind every KEM and signature algorithm to a documented wire encoding, parameter set, authentication and trust policy.
- Validate certificates, CSR issuance, signed CRLs and PKCS#12 key imports with negative fixtures.
- Run an actual TLS peer handshake, hostname/chain checks, client auth, SNI/ALPN and shutdown on target platforms.
- Where hardware custody is required, test a physical token/provider and signed attestation trust, not only compiled availability.
- Record each of the 30 direct SDK qualification sections and distinguish offline checks from multi-host interoperability.
- Publish QRCS-CASL/authorized license terms and complete distribution hashes, binary dependencies and installer smoke results.
- Do not claim unexecuted tests passed; attach exact platform/toolchain and native QSC profile to each result.

Annex A: Types, Classes and Enumerations

A.1 Core types and status values

Type	Source	Contract
qsc::byte, byte_vector, byte_array<N>	qsccpp_common.h	Binary scalar, dynamic and fixed-width byte storage.
qsc::status	qsccpp_common.h	Seven result categories; not a native QSC enum.
qsc::error_code	qsccpp_common.h	Typed errors in crypto_exception.
qsc::crypto_exception	qsccpp_exception.h	C++ failure with location, origin and message.
qsc::secure_byte_allocator	qsccpp_secure.h	Stateless secure erasure allocator; equality is allocator interchangeability.
qsc::secure_vector	qsccpp_secure.h	Move-only secret bytes; zeroizes storage but does not pin memory.
qsc::tls::peer_info	qsccpp_tls.h	Negotiated and peer certificate snapshot; no independent trust.
qsc::tls::context/connection/listener/server	qsccpp_tls.h	Native TLS RAII hierarchy.
qsc::x509::certificate/trust_store	qsccpp_x509.h	Heap-stable certificate parsing and trust store ownership.
qsc::x509::certificate_request/crl_builder/pkcs12_bundle	qsccpp_x509_issuance.h	Owns CSR, CRL and sensitive PKCS#12 native storage.
qsc::hardware::provider/key	qsccpp_hwprovider.h	Optional RAII keys and device provider, gated by QSC_HARDWARE_ENABLED.
qsc::drbg::hcg/csg	qsccpp_drbg.h	Own and clear native DRBG state; noncopyable/nonmovable.
qsc::sha2::stream256/384/512	qsccpp_sha2.h	Incremental digest state with reset/update/finalize.
qsc::sha3::stream128/256/512	qsccpp_sha3.h	Incremental SHA3 digest state.

Table A.1 C++ types and lifecycle contracts

A.2 Parameter selectors

The ECC/EdDSA/EDDH wrappers expose named curve enums; PQC modules expose appropriate parameter_set selectors; SLH-DSA exposes parameter and prehash selectors. These are C++ wrapper types, not standalone cross-platform ABI-serialized enum layouts. Applications must negotiate protocol identifiers explicitly and obtain native sizes for the exact selected profile.

Annex B: Source, Module and Test Inventory

Public C++ header	Callables	Module
qsccpp_aes.h	15	Aes
qsccpp_chacha.h	6	Chacha
qsccpp_common.h	2	Common
qsccpp_csx.h	4	Csx
qsccpp_drbg.h	14	Drbg
qsccpp_ecdh.h	23	Ecdh
qsccpp_ecdsa.h	26	Ecdsa
qsccpp_eddh.h	23	Eddh
qsccpp_eddsa.h	24	Eddsa
qsccpp_encoding.h	8	Encoding
qsccpp_entropy.h	10	Entropy
qsccpp_exception.h	6	Exception
qsccpp_falcon.h	10	Falcon
qsccpp_hkdf.h	12	Hkdf
qsccpp_hmac.h	9	Hmac
qsccpp_hqc.h	20	Hqc
qsccpp_hwprovider.h	25	Hwprovider
qsccpp_kmac.h	12	Kmac
qsccpp_mceliece.h	18	Mceliece
qsccpp_mldsa.h	20	Mldsa
qsccpp_mlkem.h	21	Mlkem
qsccpp_pbmac1.h	4	Pbmac1
qsccpp_qmac.h	2	Qmac
qsccpp_rcs.h	8	Rcs
qsccpp_scb.h	4	Scb
qsccpp_secrand.h	7	Secrand
qsccpp_secure.h	16	Secure
qsccpp_sha2.h	33	Sha2
qsccpp_sha3.h	33	Sha3
qsccpp_shake.h	20	Shake
qsccpp_slhdsa.h	10	Slhdsa
qsccpp_sphincsplus.h	10	Sphincsplus
qsccpp_tls.h	118	Tls
qsccpp_utils.h	9	Utils
qsccpp_x509.h	39	X509
qsccpp_x509_issuance.h	38	X509 Issuance

Table B.1 Extracted header declarations

B.1 Project roots and entry points

QSCCPP/ contains 37 native C++ implementation files and corresponding public headers plus Visual Studio and CMake fragments. QSCSDKCPP/ contains the executable test runner, its help and CPU information helpers, 29 named test modules and a secure-vector internal test section. The umbrella header qsccpp.h has no callable declarations of its own and collects every public header.

B.2 Source export and language boundary

The project is a C++17 library rather than a C ABI. It does not have the QSCABI 329-symbol native contract, and no function count here should be equated with QSCABI public exports. Public C++ API stability requires matching compiler ABI, standard-library object layout and QSC state compatibility. If stable cross-language linkage is required, a separately designed C ABI layer would be necessary.

Annex C: Compatibility Notes and Source Findings

The following findings should accompany any release review. They are grounded in the supplied source tree and were not derived from an independent compilation run.

- The current QSCSDKCPP console uses 29 named _run_all() module calls and one secure buffer lifecycle section, for 30 test sections.
- The SDK test runner is directly sequenced and does not use switch-based dispatch or a test registry to invoke modules.
- The supplied TLS connection-state test passes only if an unconnected object reports no handshake and no ALPN; it is not a network connection test.
- The stateless secure allocator equality always compares equal by design; this is not a cryptographic constant-time byte compare.
- The optional hardware bridge is unavailable unless compiled with QSC_HARDWARE_ENABLED; presence of that macro does not certify connected hardware.
- The CMake fragments require parent-provided QSC target definitions and external QSC sources/headers, which are absent from this ZIP.
- The source declares QSCCPP version 0.1.0; no independently verified release binary/version query was supplied.
- The companion QSCSDKCPP source contains QRCS-CASL notice. No separate complete controlling commercial license was included in this ZIP.
- The public C++ ABI is not the same API boundary as QSCABI, QSCRS or an OpenSSL compatibility provider.
- The document records source inspection only; no compilation, linked runtime, platform test, certificate trust check or live hardware verification was executed.